

Aizzen Risk Management Web Platform Refactoring Request — Product Requirements Document (PRD)

Bilingual (ქართული / English) · Draft v0.1 · React SPA (localStorage) → AWS backend roadmap

Principes:

- **Preserve Existing Functionality**

Do not remove, break, or alter existing functionality unless explicitly required. Prevent regressions and maintain backward compatibility wherever possible.

- **Maintain Product and Brand Consistency**

Do not change the product's design or user experience unless the task requires it. Any necessary design changes must follow the existing branding, design system, and interaction patterns.

- **Improve Code Safely**

Fix relevant bugs and optimize performance, reliability, and code quality where appropriate—without introducing unnecessary changes or expanding the agreed scope.

- **Build for Long-Term Maintainability**

Refactor and structure the project to support future development. Code should be modular, readable, testable, documented where necessary, and easy to extend.

- **Apply Secure Development Practices**

Treat security as a core enabler of the product. Follow established secure-development practices, including the OWASP Top 10, least privilege, secure defaults, input validation, and responsible handling of sensitive data.

- **Validate Every Change**

Every change must undergo appropriate automated testing, security checks, and quality assurance. Confirm that the requested outcome works and that existing functionality remains unaffected.

- **Deliver Large Changes Incrementally**

Break large or complex changes into clearly defined phases and small, manageable tasks. Implement and validate each step before proceeding to the next one.

1. Overview / მიმოხილვა

Aizzen Risk Management (RM) platform for maintaining a risk register, assessing risks on an inherent/residual/target basis, tracking mitigation action plans, and reporting.

.

2. Scope & Constraints

Aizzen Web Application Refactoring Request

The objective is to refactor the existing Aizzen web application in accordance with the principles outlined above.

Currently, the entire project is contained within a single HTML file. It must be reorganized into a clear, modular, and maintainable structure that makes future development, updates, and changes easier and safer.

The existing Risk Management module (**Risk Register**) and the user and access management functionality (**Administration**)—which is based on an Active Directory–like Organizational Unit (OU) structure—are largely functioning correctly. Their existing behavior and capabilities must be preserved, and the refactoring must not introduce any functional degradation or regressions.

It is also essential to preserve and improve the **Site Administration** page. Its current functionality must remain fully operational, and any improvements must be implemented without removing or compromising existing capabilities.

- Languages: Georgian + English, runtime toggle (i18n).
- Brand: navy #1A2151, dark navy #0D1128, white; cotton-flower logo top-left.
- Data model.backend-portable (no logic hard-coded to localStorage).

Data Architecture and Storage Requirements

The solution must support three interchangeable data-storage configurations:

1. **Backend-Portable Data Model**
The data-access layer must be decoupled from the storage implementation. No business logic or data-handling behavior may be hard-coded to `localStorage`. If `localStorage` is used, it must be accessed through a replaceable adapter.
2. **On-Premises Data Repository**
Provide an API-based integration with a locally hosted, on-premises data repository.
3. **AWS S3 Data Storage**
Provide a secure API-based integration for storing and retrieving data from a designated AWS S3 bucket. AWS credentials must never be exposed in the frontend.

All three configurations must use a consistent data contract and be selectable through configuration without requiring changes to the application's business logic.

3. Branding & Layout / ბრენდინგ

- Top-left: our brand logo (cotton flower).

- Top-right: uploadable CLIENT company logo (admin sets it, stored as base64 in Phase 1).
- Palette applied globally: primary #1A2151, surface #0D1128, accent white; risk colors green/amber/orange/red per Rating Matrix.
- Bottom-left corner: ADMINISTRATION entry point (per your requirement).

Logo:



4. Roles & Permissions / როლები

- Risk Owner:
- Identification of their risks
 - Prioritisation of their risks

- Ownership of their risks
- Understanding their risks
- Ensure controls are in place and are designed effectively to mitigate risk
- Ensure controls are documented
- Address remediation requirements to close controls design gaps/weaknesses
- Ensure risk and control environment details are reflected in an appropriate risk register
- Documentation, monitoring and bi-annual validation of any accepted risks
- Ensure there is a Risk Co-Ordinator* in place to assist with ongoing engagement with risk teams

Control Owner : Responsible for ensuring that controls are designed effectively and operating to

mitigate the risk. (Note the control owner and risk owner may be the same person)

- Detailed understanding of how the control mitigates the inherent risk
- Ensuring appropriate documentation is in place to demonstrate the existence and the performance of the control
- Assess and respond to any identified control deficiencies or issues raised by amending the design or operation of controls.

Action Owner : Ensuring individual or multiple actions are implemented/delivered to reduce the risk.

(Note the action owner and risk owner may be the same person)

- Address remediation actions required to close control gaps and operating effectiveness deficiencies/issues /weaknesses
- Ongoing review of actions to ensure achievement of target delivery date(s)

Risk Manager- administrates the risk management modul have all the prmision including editing and changins all risk inputs

5. Administration Functionality / ადმინისტრირება

- Add/edit/remove risk CATEGORIES (from Risk Category Library, editable).
- Add custom ATTRIBUTES to the Risk Register as COLUMNS (text, number, date, select, user-picker) — dynamic schema.
- Manage USERS: assign roles; pick risk owners/action owners FROM the user directory.
- Configure SSO (SAML) — IdP metadata, ACS URL, role mapping from SAML attributes.
- Upload/replace client logo (top-right).
- Configure Risk Rating Matrix (likelihood × impact scale, thresholds, colors).

8. Test Users (3) / სატესტო იუზერები

Passwords are demo-only.

0. Supper-Admin.local has access to front page management and hath access to risk management modul as well, admin aceee.

1 admin@erm.local / Admin — role: Administrator — Pass: Admin#2026

S.phikidze@aizzen.com – administrator- Pass: Admin#2026

d.baghdavadze@aizzen.com- administrator- Pass: Admin#2026

9. Auth Test Plan / ავტორიზაციის ტესტი

- Login with each user → correct role loaded into session.
- Auditor: all edit/save buttons hidden & API-guarded (view-only verified).
- Risk Owner: can edit own risks only, not others.
- Admin: sees Administration panel (bottom-left); others do not.
- Invalid credentials → error; session persists on refresh (localStorage token).

10. Autorization

Implement Google Sign-In with Internal User Authorization

Implement secure **Sign in with Google** functionality in the existing web application. Google must be used only to authenticate the user's identity. Access to the application, roles, and permissions must continue to be controlled by the existing internal user-management system.

Required Authorization Logic

1. Administrators must continue creating and managing users through the existing Administration panel.
2. Each internal user record must contain:
 - Email address
 - Account status, such as active or disabled
 - Assigned role
 - Assigned permissions
 - Google account subject identifier (`sub`) after the account is securely linked
3. A user may access the application through Google only when:
 - Google authentication succeeds.
 - The returned ID token is valid and verified on the server.
 - The Google account's `email_verified` value is `true`.
 - An active internal user with the same normalized email address already exists.
 - The internal user has permission to access the application.
4. Do not automatically create users after Google authentication. Users must first be provisioned by an administrator.
5. Do not grant access merely because the email belongs to the `aizzen.com` domain. A matching, active internal user account must always exist.
6. On the first successful login, match the verified Google email address to the internal user and securely store Google's stable `sub` identifier. Use the linked `sub` identifier for subsequent identity checks while continuing to verify that the internal account is active.
7. Roles and permissions must always be loaded from the internal authorization system. Never derive or replace application permissions using Google profile data.

Example

If an administrator creates the following user:

- Email: `d.baghdavadze@aizzen.com`
- Status: Active
- Role: Administrator
- Permissions: Assigned through the existing Administration module

The owner of that Google account must be able to select **Sign in with Google** and access the application with the Administrator role and its assigned permissions.

If the email does not exist in the internal user directory, the account is disabled, or Google does not verify the email, access must be denied.

Security Requirements

- Use the officially supported Google Identity Services and OpenID Connect flow appropriate for the application's architecture.

- Prefer the Authorization Code flow with PKCE and server-side token validation.
- Validate the token signature, issuer, audience, expiration time, nonce, and other required claims.
- Protect the authentication flow against CSRF and replay attacks using `state`, `nonce`, and PKCE where applicable.
- Never expose the Google client secret, access tokens, or sensitive configuration in frontend code.
- Store secrets in secure environment variables.
- Create application sessions using secure, `HttpOnly`, `Secure`, and appropriately configured `SameSite` cookies.
- Prevent session fixation and rotate the session identifier after login.
- Use generic access-denied messages that do not reveal whether an email is registered.
- Record successful logins, failed attempts, account-linking events, and access denials in the security audit log.
- Apply appropriate login rate limiting.
- Do not store authentication tokens or sensitive session data in `localStorage`.

User Interface Requirements

- Add a **Sign in with Google** button to the existing login page.
- Preserve the application's current design system and branding.
- Display clear but non-sensitive error messages for unsuccessful login attempts.
- Do not remove or degrade any existing authentication or Administration functionality.

Implementation Process

Before making changes:

1. Inspect the existing authentication architecture, user schema, permission model, API layer, and session-management logic.
2. Identify potential compatibility and security risks.
3. Present a short implementation plan.
4. Implement the change in small, testable phases.
5. Avoid unrelated refactoring or design changes.

Acceptance Criteria

The implementation is complete only when the following scenarios are tested:

- An active, pre-provisioned user with the correct verified Google email can sign in.
- The user receives exactly the role and permissions configured internally.
- An unknown Google account is denied access.
- A disabled internal user is denied access.
- An unverified Google email is denied access.
- Invalid, expired, replayed, or incorrectly issued tokens are rejected.
- Changing or removing an internal user's permissions takes effect without changing their Google account.
- Disabling or deleting an internal user immediately prevents future access.
- Existing authentication, Administration, Risk Register, and Site Administration functionality continues to work without regression.
- Relevant automated tests, security checks, and QA validation pass.

• Addendum - Old Aizen Risk Management solution documentation.

The documentation below (Addendum - Old Aizen Risk Management solution documentation.) was created during the project's initial development phase and is provided to help you understand the product, its functionality, and its intended behavior. If any part of this documentation conflicts with Requirements 1–10 above, the requirements above must take precedence!!!

AIZEN Risk & Compliance Risk Management and Risk Administration

ფუნქციური და ლოგიკური სახელმძღვანელო

პროდუქტის ვერსია: 4.1.0

დოკუმენტის ვერსია: 1.0

სტატუსი: As-built Phase 1 + Phase 2 backend-ready requirements

ენა: ქართული / English system terms

თარიღი: 2026

ეს სახელმძღვანელო განკუთვნილია Product Owner-ისთვის, Risk Team-ისთვის, Administrator-ისთვის, Frontend/Backend/QA გუნდებისთვის და Auditor/Information Security stakeholders-ისთვის.

დოკუმენტის გამოყენება

დოკუმენტი ოთხ ნაწილადაა დაყოფილი:

1. Risk Management Module - end-to-end business workflow და თითოეული ფუნქციის logic;
2. Risk Administration Module - master data, hierarchy, users, roles, matrix და data tools;
3. Cross-Module Architecture - RBAC, repository, audit, migration და Phase 2 backend;
4. Source Traceability - uploaded policies, framework, matrix და change requests-ის კავშირი.

Status labels:

- **Implemented - Phase 1:** მიმდინარე local/offline build-ში მუშაობს;
- **Business Rule:** უნდა დარჩეს უცვლელი storage/backend-ის მიუხედავად;
- **Phase 2 Requirement:** production backend-ზე server-side უნდა აღსრულდეს;
- **Known Limitation:** მიმდინარე browser prototype-ის საზღვარი.

AIZEN Risk Management Module - ფუნქციური და ლოგიკური

სპეციფიკაცია

დოკუმენტის ტიპი: Functional Specification / Business Logic Manual

პროდუქტის ვერსია: AIZEN Risk & Compliance 4.1.0

დოკუმენტის ვერსია: 1.0

სტატუსი: As-built Phase 1 specification with Phase 2 requirements

მიზნობრივი აუდიტორია: Risk Team, Product Owner, Frontend, Backend, QA, Audit, Information Security

1. დოკუმენტის მიზანი

ეს დოკუმენტი აღწერს Risk Management მოდულის სრულ ფუნქციურ ლოგიკას ისე, რომ გუნდის წევრმა შეძლოს გაიგოს:

- რა ბიზნეს-მიზანს ემსახურება თითოეული ეკრანი;
- როგორ იქმნება, ფასდება, მუშავდება და მონიტორინგდება რისკი;
- როგორ გამოითვლება Inherent, Residual და Target risk;
- როგორ მუშაობს Controls, Remediation Actions და Risk Acceptance;
- როგორ მოქმედებს Role-Based და record-level access;
- რა ინახება მონაცემთა მოდელში და როდის იქმნება history/audit event;
- როგორ მუშაობს Dashboard და Report Builder;
- რომელი წესი არის უკვე იმპლემენტირებული და რომელი უნდა გახდეს backend-ის პასუხისმგებლობა.

Risk Management lifecycle ეფუძნება ოთხ ეტაპს: Identification, Assessment, Response და Monitoring/Reporting/Continuous Review. რისკის ჩანაწერში საჭიროა risk owner, controls/control owner და საჭიროებისას action owner; შეფასება კეთდება Inherent, Residual და Target მდგომარეობებზე. ეს მიდგომა ასახულია Enterprise Risk Management Framework-ის lifecycle ნაწილში.

2. მოდულის საზღვრები

2.1 მოდულში შედის

- Dashboard Builder;
- Risk Register;
- Risk creation and editing;
- Individual Risk View;
- 5x5 Risk Assessment;
- Existing Controls;
- Remediation Actions;
- Risk Acceptance;
- Assessment History;
- Local Audit Trail;
- Saved Register Views;

- CSV/Excel export;
- Report Template Builder;
- bilingual UI - English/Georgian;
- role and Business Unit scope-aware data visibility.

2.2 მოდულში არ შედის მიმდინარე Phase 1-ში

- server-side database;
- real SAML/OIDC authentication;
- immutable audit ledger;
- email notifications;
- approval workflow engine;
- file/evidence repository;
- concurrent editing/conflict detection;
- automatic reminder scheduler;
- API-enforced permissions.

2.3 ძირითადი routes

Route	დანიშნულება	მინიმალური permission
/app/dashboard	Dashboard-ების ნახვა და, უფლების შემთხვევაში, მართვა	dashboard: read
/app/register	Risk Register	register: read და risks: read
/app/risks/:id	Individual Risk View	Risk record visibility
/app/reports	Report Template Library და generated reports	reports: read
/app/administration	Risk Administration	Administrator/Super Administrator

3. ERM ბიზნეს-ცნებები

3.1 Risk

რისკი არის მომავალი გაურკვეველი მოვლენა ან გარემოება, რომელმაც შეიძლება უარყოფითად იმოქმედოს ბიზნესის მიღწევაზე. სისტემაში risk record არ არის incident/issue record; მატერიალიზებული მოვლენის ცალკე issue-management მოდული ჯერ არ არსებობს.

3.2 Current Risk და Emerging Risk

- **Current:** მიმდინარე ან უკვე იდენტიფიცირებული exposure, რომლის შეფასება შესაძლებელია არსებული ინფორმაციით.
- **Emerging:** ახალი ან მომავალი exposure, რომლის სრული გავლენა ან ალბათობა ჯერ არ არის საიმედოდ ცნობილი.

3.3 Inherent Risk

კონტროლების გათვალისწინებამდე არსებული რისკის დონე. შეფასებისას მომხმარებელი აკეთებს დაშვებას, რომ კონტროლი არ არსებობს ან არ მუშაობს.

3.4 Residual Risk

არსებული კონტროლებისა და მათი მოქმედების ეფექტურობის გათვალისწინებით დარჩენილი მიმდინარე რისკის დონე.

3.5 Target Risk

რეალისტურად სასურველი risk level, რომლის მიღწევაც მოსალოდნელია კონტროლებისა და remediation action-ების წარმატებით განხორციელების შემდეგ.

3.6 Control

კონკრეტული მოქმედება ან მექანიზმი, რომელიც ამცირებს risk likelihood-ს, impact-ს ან ორივეს. კონტროლი სისტემაში ხელით იქმნება და არ მოითხოვს წინასწარ განსაზღვრული control library-დან არჩევას.

3.7 Remediation Action

დაგეგმილი ქმედება, რომელიც გამოიყენება მაშინ, როდესაც:

- კონტროლი არ არსებობს;
- კონტროლის დიზაინი არასაკმარისია;
- კონტროლი არ მუშაობს ეფექტურად;
- Residual risk Target risk-ზე მაღალია;
- management decision ითხოვს დამატებით mitigation-ს.

3.8 Risk Response

სისტემა მხარს უჭერს ოთხ პასუხს:

- **Avoid** - საკმინაობის შეწყვეტა ან შეცვლა, რათა exposure მოიხსნას;
- **Mitigate** - likelihood/impact-ის შემცირება კონტროლებით ან actions-ით;
- **Transfer** - risk-ის ნაწილის გადაცემა/გაზიარება, მაგალითად outsourcing ან insurance;
- **Accept** - მიმდინარე exposure-ის ფორმალური მიღება დამატებითი remediation-ის გარეშე.

3.9 Risk Trend, Direction to Target და Outlook

ეს სამი მაჩვენებელი ერთმანეთისგან განსხვავდება:

- **Historical Trend:** ბოლო ორ assessment-ში Residual score-ის ცვლილება.
- **Direction to Target:** მიმდინარე Residual score-ის Target score-თან შედარება.
- **Risk Outlook:** Risk Owner/Risk Manager-ის ხელით არჩეული მომდევნო 12 თვის მოლოდინი.

4. Risk Management Lifecycle და სისტემური workflow

4.1 Stage 1 - Identification

მომხმარებელი ქმნის risk record-ს და ავსებს:

1. Risk ID/reference;
2. Risk Type;
3. Risk Title;
4. Category;
5. Business Unit;
6. Risk Owner;
7. Origin Date;
8. Cause;
9. Event;

10. Consequence;
11. Status narrative;
12. საწყის Inherent/Residual/Target assessment-ს.

Business rule: Cause, Event და Consequence სავალდებულოა. ბუნდოვანი ერთი აზრის ნაცვლად risk description იყოფა სამ ნაწილად, რათა mitigation და reporting კონკრეტული იყოს.

4.2 Stage 2 - Assessment

ყოველი risk ფასდება სამ მდგომარეობაში:

Inherent = controls-მდე
Residual = არსებული controls-ის შემდეგ
Target = desired future state

თითოეულ მდგომარეობაზე ირჩევა:

- Impact: 1-5;
- Likelihood: 1-5;
- Score: Impact x Likelihood;
- Rating: configured 5x5 matrix cell.

4.3 Stage 3 - Response

მომხმარებელი ირჩევს Avoid, Mitigate, Transfer ან Accept პასუხს.

- Mitigate-ისას პრაქტიკულად მოსალოდნელია controls/actions-ის არსებობა.
- Accept-ისას იხსნება acceptance details და rationale სავალდებულოა.
- Accepted status-ის შენახვისას საჭიროა approver, approval date და valid until.

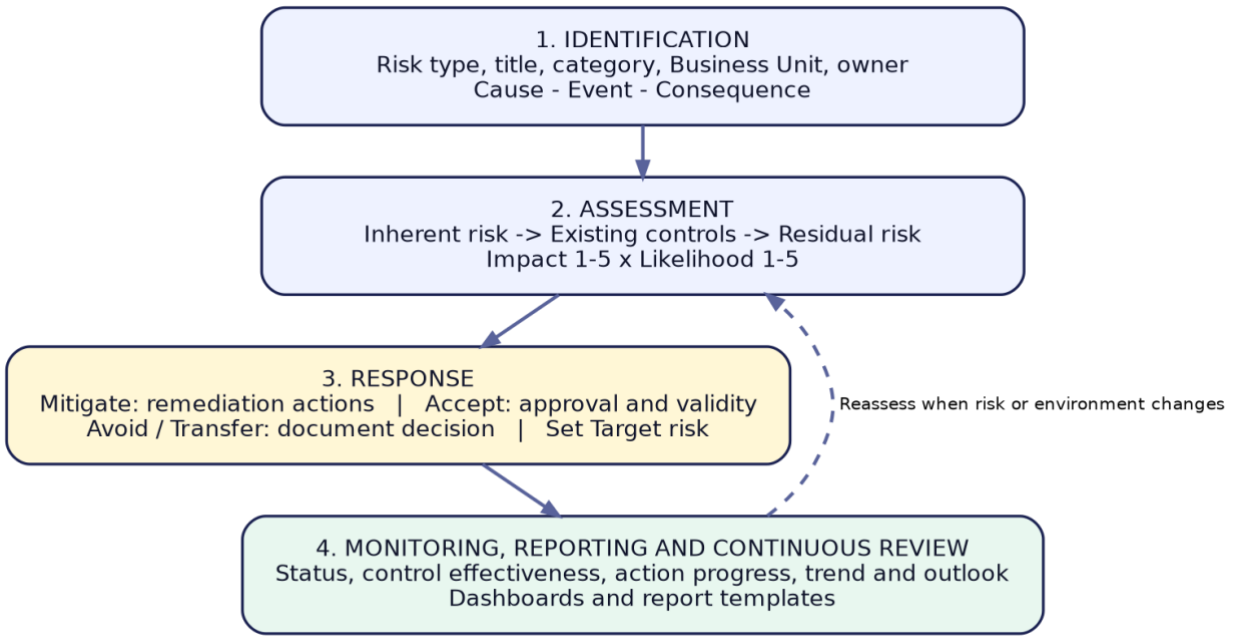
4.4 Stage 4 - Monitoring, Reporting & Continuous Review

Risk Owner/Risk Manager განაახლებს:

- risk status-ს;
- review date-ს;
- status narrative-ს;
- assessment-ს;
- control effectiveness-ს;
- action progress/status/deadline-ს;
- outlook-ს;
- acceptance review date-ს.

Dashboard, Risk Register და Reports კითხულობს იმავე risk dataset-ს, ამიტომ update ავტომატურად აისახება ყველა reporting view-ში.

4.5 მაღალი დონის flow



Risk Management lifecycle

5. Risk Data Model

5.1 Risk entity

Field	Type	აღწერა	მთავარი წესი
id	string	ტექნიკური უნიკალური ID	risk_<timestamp>_<random>
ref	string	ბიზნეს reference	Business Unit code + 3-digit sequence
title	string	უნიკალური/აღწერილობითი სათაური	Save-ზე trim; required
type	Current / Emerging	risk type	manager-level field
categoryId	string	category reference	active/current category
businessUnitId	string	ownership scope	user access-ს უნდა შეესაბამებოდეს
riskOwnerId	string	accountable owner	active and BU-scoped user
originDate	date	პირველად იდენტიფიცირების თარიღი	default: today
reviewDate	date	შემდეგი formal review	default: today + 12 months
targetDate	date	target state/action horizon	default: today + 6 months
status	enum	workflow status	manager controlled
responseType	enum	Avoid/Mitigate/Transfer/Accept	Risk Owner ან manager
outlook	enum	Increasing/Stable/Decreasing	manual 12-month outlook
cause	text	risk source/root cause	required

Field	Type	აღწერა	მთავარი წესი
event	text	შესაძლო მოვლენა	required
consequence	text	შესაძლო შედეგი	required
statusNarrative	text	მიმდინარე მდგომარეობის ნარატივი	monitoring/reporting
inherent	score object	controls-მდე score	Impact/Likelihood 1-5
residual	score object	current score	Impact/Likelihood 1-5
target	score object	desired score	Impact/Likelihood 1-5
controls	array	structured controls	empty allowed
actions	array	remediation actions	empty allowed
acceptance	object	acceptance metadata	required when response/status implies acceptance
custom	object	custom attribute values	keyed by Attribute ID
history	array	assessment snapshots	initial + score changes
audit	array	risk-specific events	newest first
updatedAt	datetime	last save timestamp	updated on save

5.2 Score object

```
{
  impact: 1 | 2 | 3 | 4 | 5;
  likelihood: 1 | 2 | 3 | 4 | 5;
}
```

Score და rating მონაცემთა მოდელში ცალკე არ იწახება. ისინი ითვლება runtime-ზე, რათა matrix configuration-ის ცვლილება ყველა view-ზე ერთნაირად აისახოს.

5.3 Control object

Field	აღწერა
id	Control-ის ტექნიკური ID
title	Control title; ცარიელი title-ის მქონე control Save-ზე იშლება
ownerId	Control Owner
performer	ვინ ასრულებს კონტროლს
description	როგორ მუშაობს კონტროლი
frequency	Continuous/Daily/Weekly/Monthly/Quarterly/სხვა ტექსტი
intendedOutcome	რატომ არსებობს კონტროლი და რა შედეგს ელოდება
evidenceLocation	evidence-ის მდებარეობის ტექსტური reference
keyControl	boolean; target level-ისთვის კრიტიკული control
type	Directive/Preventative/Detective/Corrective
automation	Manual/Automated/Semi-Automated
status	Effective/Needs Improvement/Ineffective/Not Assessed

5.4 Remediation Action object

Field	აღწერა
id	Action technical ID
title	Action name; empty title Save-ზე იშლება
description	შესასრულებელი სამუშაო
deliverable	დასრულებისას მისაღები შედეგი/evidence
ownerId	Action Owner
dueDate	Target delivery date
status	Not Started/In Progress/Blocked/Rescheduled/Overdue/Completed
priority	Low/Medium/High/Critical
progress	0-100, step 5
notes	დამატებითი კომენტარი

5.5 Acceptance object

Field	აღწერა	მიმდინარე წვდომა
rationale	მიღების დასაბუთება	Risk Owner/Manager
initiatorId	acceptance initiator	Risk Owner/Manager
approverId	approving user	Manager/Admin
approvalDate	approval date	Manager/Admin
validUntil	acceptance expiry	Manager/Admin
reviewDate	next review	Risk Owner/Manager

5.6 Assessment History item

```
{
  id: string;
  date: YYYY-MM-DD;
  inherent: Score;
  residual: Score;
  target: Score;
  note: string;
  actorId: string;
}
```

5.7 Audit event

```
{
  id: string;
  date: ISO datetime;
  actorId: string;
  action: string;
  entityType: string;
  entityId: string;
  summary: string;
  changes?: string[];
}
```


6. Risk ID generation logic

6.1 ფორმატი

<Business Unit Code>-<3 digit sequence>

მაგალითები:

TECH-001
SEC-004
LRC-002

6.2 ალგორითმი

1. აიღე არჩეული Business Unit-ის code და გადააკეთე uppercase-ად.
2. მოძებნე ყველა არსებული risk, რომლის ref იწყება იგივე prefix-ით.
3. გამოყავი ბოლო numeric ნაწილი.
4. აიღე მაქსიმალური ნომერი.
5. დაამატე 1.
6. შეავსე მარცხნიდან ნულებით 3 სიმბოლომდე.

```
next = max(existingNumbers) + 1  
ref = prefix + '-' + padStart(next, 3, '0')
```

6.3 edge cases

- თუ Business Unit code ცარიელია, fallback prefix არის ERM.
- არსებული risk-ის reference ავტომატურად არ იცვლება Business Unit-ის შემდგომი ცვლილებისას.
- ახალი risk-ის editor-ში Business Unit-ის შეცვლისას reference ხელახლა ითვლება.
- sequence reuse არ ხდება წაშლის/არქივის შემდეგ; მიმდინარე Phase 1-ში risk delete ფუნქცია საერთოდ არ არის.

7. Default values ახალი risk-ისთვის

Field	Default
Business Unit	მომხმარებლის პირდაპირი scope-ის პირველი active unit; fallback effective scope/first active unit
Category	პირველი active category
Risk Owner	მიმდინარე Risk Owner user; სხვაგვარად პირველი active Risk Owner; fallback current user
Origin Date	Today
Review Date	Today + 12 months
Target Date	Today + 6 months
Status	Draft
Risk Type	Current
Response	Mitigate
Outlook	Stable
Inherent	Impact 3 x Likelihood 3
Residual	Impact 2 x Likelihood 3

Field	Default
Target	Impact 2 x Likelihood 2
Controls	empty
Actions	empty
Custom number	0
Other custom values	empty string

8. Risk creation and update validation

8.1 Required validations

Save იბლოკება, თუ:

1. title ცარიელია;
2. categoryId არ არის არჩეული;
3. riskOwnerId არ არის არჩეული;
4. Cause, Event ან Consequence ცარიელია;
5. Response = Accept და acceptance rationale ცარიელია;
6. Status = Accepted და არ არის approver, approval date ან valid until.

8.2 data cleaning

Save-ის წინ:

- title იწმინდება leading/trailing spaces-ისგან;
- ცარიელი reference იცვლება generated reference-ით;
- updatedAt ახლდება;
- ცარიელი title-ის controls/actions იფილტრება;
- custom values ინარჩუნებს Attribute ID-ზე მიბმულობას.

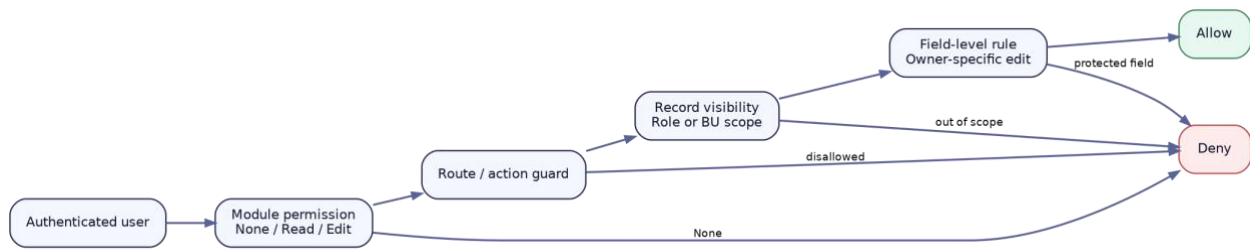
8.3 history creation

- ახალი risk ქმნის Initial assessment history item-ს.
- არსებული risk-ის save ქმნის ახალ history item-ს მხოლოდ მაშინ, თუ შეიცვალა Inherent, Residual ან Target score object.
- მხოლოდ title/status/control/action change history item-ს არ ქმნის, თუმცა audit event იქმნება.

8.4 audit creation

- New risk: risk.created.
- Update: risk.updated.
- Update event-ის changes შეიცავს changed master fields-ს, score transition-ს, controls/actions change-ს ან Record saved fallback-ს.

9. Role-based access და record-level visibility



Access decision sequence

9.1 Module permission levels

თითოეულ მოდულზე role-ს შეიძლება ჰქონდეს ერთ-ერთი დონე:

Permission	შედეგი
None	მოდული არ ჩანს; შესაბამის route-ზე წვდომა იბლოკება
Read	მონაცემის ნახვა შესაძლებელია; create/edit/delete მოქმედებები იბლოკება
Edit	role-ს შეუძლია მოდულის ფარგლებში დაშვებული ცვლილებები

თუ მომხმარებელს რამდენიმე role აქვს, module permission ითვლება ყველაზე მაღალი დონით:

None < Read < Edit

9.2 Risk visibility by role

Role	რომელი risks ჩანს
Super Administrator	ყველა risk
Administrator	ყველა risk
Risk Manager	ყველა risk
Auditor	ყველა risk, read-only
Risk Owner	მხოლოდ riskOwnerId = currentUser.id
Control Owner	მხოლოდ ის risks, სადაც მინიმუმ ერთი control-ის ownerId = currentUser.id
Action Owner	მხოლოდ ის risks, სადაც მინიმუმ ერთი action-ის ownerId = currentUser.id
Custom role	მომხმარებლის effective Business Unit scope-ში არსებული risks

9.3 Business Unit effective scope

Business Unit-ის პირდაპირი მინიჭება მომხმარებელს აძლევს წვდომას:

- მინიჭებულ unit-ზე;
- მის ყველა descendant unit-ზე;
- მომავალში ამ subtree-ში დამატებულ ახალ descendant unit-ებზე.

Parent unit-ზე წვდომა არ მემკვიდრეობს ქვემოდან ზემოთ. მაგალითად, Information Security-ზე მინიჭებული user ვერ ხედავს Technology Division-ის parent-level ან IT Operations sibling risks-ს.

9.4 Record edit rules

Role	რისი შეცვლა შეუძლია
------	---------------------

Role	რისი შეცვლა შეუძლია
Administrator / Super Administrator / Risk Manager	სრული risk record: master data, owner, status, assessments, controls, actions, acceptance, custom fields
Risk Owner	მხოლოდ მისთვის მინიჭებული risk-ის assessment, narrative, dates, response/outlook, controls, actions და custom fields; manager-only master fields/owner/status შეზღუდულია
Control Owner	მხოლოდ საკუთარი controls; Save-ისას სხვა risk fields საწყის მნიშვნელობაზე ბრუნდება
Action Owner	მხოლოდ საკუთარი actions; Save-ისას სხვა risk fields საწყის მნიშვნელობაზე ბრუნდება
Auditor	ცვლილება აკრძალულია

9.5 Known Phase 1 limitation - multiple scoped roles

მიმდინარე client logic named-role branches-ს რიგით ამოწმებს. თუ ერთ user-ს ერთდროულად აქვს, მაგალითად, Risk Owner და Control Owner, visibility შეიძლება პირველი matched role-ის წესით განისაზღვროს და არა ყველა scope-ის union-ით.

Phase 2 requirement: backend authorization-მა record-level grants უნდა გააერთიანოს union პრინციპით და თითოეული permission გადაწყვეტილება server-side დააფიქსიროს.

10. Risk Register

10.1 მიზანი

Risk Register არის მოდულის მთავარი სამუშაო სია. იგი აერთიანებს risk ownership-ს, assessments-ს, controls-ს, response/action plan-ს, status-ს, trend/outlook-სა და custom attributes-ს.

10.2 მონაცემების წყარო

Register იღებს მხოლოდ visibleRisks(currentUser) შედეგს. შემდეგ ამ dataset-ზე გამოიყენება search, filters და sorting. UI-ში hidden risk-ის მოძებნა URL-ის ან search-ის საშუალებით შეუძლებელია.

10.3 Search logic

Search არის case-insensitive და ეძებს შემდეგ ველებში:

- Risk reference;
- Risk title;
- Cause;
- Event;
- Consequence;
- Category name;
- Business Unit name და სრული hierarchy path;
- Risk Owner name;
- Remediation Action title.

10.4 Filters

Filter	ქვევა
Risk Category	Level 1 category/group-ის მიხედვით
Business Unit	არჩეული unit + ყველა descendant
Residual Rating	configured matrix-ით დათვლილი Low/Medium/High/Significant
Risk Status	exact match
Risk Outlook	Increasing/Stable/Decreasing

Dashboard და Report section filters დამატებით იყენებს Risk Type-სა და Risk Owner-ს.

10.5 Sorting

მხარდაჭერილია:

- Risk title;
- Risk Owner;
- Residual numeric score;
- Target Date;
- default: Risk Reference ascending.

Header-ის ხელახალი click ცვლის მიმართულებას ascending <-> descending.

10.6 Compact და Detailed view

- **Compact:** მოკლე management list; ტექსტი და chips უფრო შეკუმშულია.
- **Detailed:** Cause/Event/Consequence, controls/action details და სრული rating chip-ები.

Detailed view-ში Inherent, Residual და Target rating chip-ები ერთნაირი ზომისაა და სრულად აჩვენებს:

Score | Rating | Impact x Likelihood

10.7 Column visibility

Columns panel-იდან მომხმარებელი ირჩევს, რომელი columns გამოჩნდეს. Active custom attribute, რომელსაც showInRegister = true, ავტომატურად ემატება selectable columns-ში.

10.8 Saved Views

Saved View ინახავს:

- search/filter state-ს;
- sort state-ს;
- visible columns-ს;
- Compact/Detailed რეჟიმს;
- user ownership-ს;
- isDefault მდგომარეობას.

ხელმისაწვდომი მოქმედებები:

1. Save current view;
2. Apply view;
3. Set as default;

4. Clear default;
5. Remove view.

Business rules:

- თითო user-ს შეიძლება ჰქონდეს მხოლოდ ერთი default view;
- ახალი default-ის დაყენება ძველ default-ს ავტომატურად ხსნის;
- Register-ის გახსნისას current user-ის default view ავტომატურად გამოიყენება;
- default view-ის წაშლის შემდეგ Register რჩება მიმდინარე/საბაზისო state-ზე;
- Saved Views user-specific-ია და სხვა user-ს არ უჩანს.

10.9 Export

CSV

CSV export მოიცავს current filtered dataset-ს და ფართო risk information-ს. Values იწმინდება/escape-დება comma, quote და new-line სიმბოლოებისთვის.

Excel-compatible export

SpreadsheetML-based .xls export ქმნის Excel-ში გასახსნელ ცხრილს. Export-ში შედის active custom attributes-იც.

Phase 1 note: export სრულდება browser-ში და შეიძლება შეიცავდეს მიმდინარე user-ისთვის ხილულ sensitive data-ს. Phase 2-ში data.exported audit event და server-side authorization სავალდებულოა.

10.10 Empty and no-result states

- თუ user-ს scope-ში risk არ აქვს - ჩანს role-aware empty state;
- თუ filters/search აბრუნებს 0 შედეგს - ჩანს clear-filter action;
- New risk დილაკი ჩანს მხოლოდ user-ს, ვისაც create უფლება აქვს.

11. Risk Editor

11.1 Tabs

Risk Editor დაყოფილია ექვს tab-ად:

1. **Basic** - reference, type, category, Business Unit, owner, dates, status, response, outlook;
2. **Structured Description** - Cause, Event, Consequence, status narrative;
3. **Assessments** - Inherent, Residual, Target;
4. **Controls** - manual structured controls;
5. **Actions** - remediation actions;
6. **Custom Fields** - active custom attributes.

11.2 New risk access

New risk შექმნა შეუძლია მხოლოდ:

- Super Administrator;
- Administrator;
- Risk Manager;

და მხოლოდ მაშინ, როდესაც risks: edit permission მოქმედებს.

11.3 Owner candidate logic

Risk Owner, Control Owner და Action Owner dropdown-ები უნდა აჩვენებდეს:

- Active users-ს;
- შესაბამის role/function capability-ს;
- არჩეული Risk Business Unit-ის effective scope-ში მყოფ users-ს.

11.4 Manager-only fields

მიმდინარე UI-ში Risk Owner-ისთვის ჩაკეტილია manager-level master data, მათ შორის owner/status-ის შეცვლა. Full editor ხელმისაწვდომია Risk Manager/Admin roles-სთვის.

11.5 Save workflow

```
Open editor
-> clone current Risk
-> apply field edits locally
-> validate required fields
-> clean controls/actions
-> compare assessment objects
-> append history if score changed
-> append audit event
-> repository.save(state)
-> close modal / refresh query state
```

11.6 Cancel

Cancel ან modal close არ უნდა persist-დებოდეს. Editor მუშაობს cloned draft-ზე; repository update მხოლოდ Save-ის წარმატების შემდეგ ხდება.

12. Risk Assessment Engine

12.1 Inputs

თითო assessment-ზე მომხმარებელი ირჩევს:

- Impact: 1-5;
- Likelihood: 1-5.

12.2 Calculation

$\text{Risk Score} = \text{Impact} \times \text{Likelihood}$

მაგალითები:

Impact 2 x Likelihood 3 = 6 -> Medium
Impact 5 x Likelihood 4 = 20 -> Significant

12.3 Impact labels

Score	English	ქართული UI მნიშვნელობა
1	Minor	მცირე
2	Moderate	საშუალო
3	Major	დიდი
4	Severe	მწვავე

Score	English	ქართული SI მნიშვნელობა
5	Critical	კრიტიკული

Impact ფასდება მინიმუმ ოთხ განზომილებაში:

- Financial;
- Reputational;
- Operational - Technology, Process, People;
- Regulatory and Legal.

თუ რამდენიმე category განსხვავებულ დონეს იძლევა, საბოლოო Impact უნდა იყოს ყველაზე მაღალი შესაბამისი score.

12.4 Likelihood labels - next 12 months

Score	Label	Probability
1	Remote	0%-5%
2	Unlikely	6%-35%
3	Possible	36%-65%
4	Likely	66%-95%
5	Almost Certain	96%-100%

12.5 Default 2026 rating matrix

Impact Likelihood	1 Remote	2 Unlikely	3 Possible	4 Likely	5 Almost Certain
5 Critical	Medium	High	High	Significant	Significant
4 Severe	Low	Medium	High	High	Significant
3 Major	Low	Medium	Medium	High	High
2 Moderate	Low	Low	Medium	Medium	High
1 Minor	Low	Low	Low	Low	Medium

Default colors:

Rating	Color	Guidance
Low	#00B050	periodic monitoring; no immediate action
Medium	#FFF200	control plan required
High	#FFB900	immediate attention and robust mitigation
Significant	#F32121	immediate and ongoing attention plus escalation

12.6 Exact-cell rule

Rating არ გამოითვლება მხოლოდ score range-ით. სისტემა იღებს selected impact/likelihood intersection cell-ს configured 25-cell matrix-იდან. ამიტომ ერთი და იგივე score სხვადასხვა cells-ში, matrix configuration-ის ცვლილების შემთხვევაში, შეიძლება განსხვავებულ rating-ს დაუკავშირდეს.

12.7 Runtime behavior

- Score და rating derived values-ია;

- Risk entity-ში ინახება მხოლოდ Impact და Likelihood;
- matrix/configuration-ის ცვლილების შემდეგ chips, heatmap, filters და reports ერთიანად recalculation-ს აკეთებს;
- all three assessment types იყენებს ერთსა და იმავე engine-ს.

12.8 Phase 2 rule

Frontend score მხოლოდ preview უნდა იყოს. API-მ დამოუკიდებლად უნდა გადაამოწმოს Impact, Likelihood, matrix version და rating; approved historical assessment-მა უნდა შეინარჩუნოს გამოყენებული matrix version.

13. Existing Controls

13.1 Manual entry principle

Controls წინასწარ განსაზღვრული catalogue-დან არ ირჩევა. თითო control ხელით იქმნება კონკრეტული risk context-ისთვის.

13.2 Quick input

Quick textarea-ში თითო არაცარიელი line ქმნის ახალ structured control card-ს.

Rules:

- leading/trailing spaces იწმინდება;
- duplicate title case-insensitive შედარებით არ ემატება;
- შექმნილი control იღებს default values-ს;
- შემდგომ user შეუძლია ყველა field-ის შეცვლა card-ში.

13.3 Control documentation minimum

Framework-ის მიხედვით control record უნდა პასუხობდეს კითხვებს:

1. ვინ ფლობს კონტროლს?
2. ვინ ასრულებს მას?
3. რას მოიცავს კონტროლი?
4. რა სიხშირით მუშაობს?
5. რა არის intended outcome?
6. სად ინახება evidence?
7. არის თუ არა Key Control?

13.4 Control types

- Directive;
- Preventative;
- Detective;
- Corrective.

13.5 Automation types

- Manual;
- Automated;
- Semi-Automated.

13.6 Effectiveness statuses

- Effective;
- Needs Improvement;
- Ineffective;
- Not Assessed.

13.7 Control Owner permissions

Control Owner user:

- ხედავს risk-ს, თუ მასში მისთვის მინიჭებული control არსებობს;
- შეუძლია მხოლოდ საკუთარი control card-ის შეცვლა;
- ვერ ცვლის risk master fields-ს, assessments-ს ან სხვა owner-ის controls-ს;
- save logic აერთიანებს მხოლოდ მის მიერ ნებადართულ control changes-ს.

13.8 Deletion

ცარიელი title-ის მქონე control Save-ზე იფილტრება. UI remove action აშორებს draft list-იდან. Phase 1-ში hard-delete history ცალკე version table-ში არ ინახება; ცვლილება audit summary-ში აისახება.

14. Remediation Actions

14.1 შექმნის პირობა

Action plan საჭიროა, როდესაც:

- control არ არსებობს;
- control enhancement საჭიროა;
- control effectiveness არასაკმარისია;
- Residual score Target score-ზე მაღალია;
- risk response-ს კონკრეტული delivery სჭირდება.

14.2 Default action

ახალი action ჩვეულებრივ იღებს:

- due date: today + 3 months;
- status: Not Started;
- priority: Medium;
- progress: 0;
- owner: შესაბამისი Action Owner, fallback Risk Owner.

14.3 Status logic

Supported statuses:

- Not Started;
- In Progress;
- Blocked;
- Rescheduled;
- Overdue;

- Completed.

14.4 Computed overdue display

Action ითვლება overdue-დ, თუ:

```
dueDate < today AND status != Completed
```

UI შეიძლება აჩვენებდეს Overdue badge-ს, თუნდაც stored status სხვა open status იყოს. Phase 2 backend scheduler/derived view-მა ეს წესი ცენტრალურად უნდა აღასრულოს.

14.5 Progress

Progress არის 0-100 და იცვლება 5-point step-ით. Completed status-ისას recommended rule არის 100%, თუმცა მიმდინარე Phase 1 არ აიძულებს ამ თანხვედრას.

14.6 Action Owner permissions

Action Owner:

- ხედავს risk-ს, თუ მასში საკუთარი action აქვს;
- ცვლის მხოლოდ თავის actions-ს;
- ვერ ცვლის risk assessment/master data-ს ან სხვა owner-ის action-ს;
- უნდა აკონტროლებდეს deliverable-ს, deadline-სა და progress-ს.

14.7 Overview table

Individual Risk Overview-ზე action summary აჩვენებს:

- Action;
- Description;
- Deliverable;
- Status;
- Deadline;
- Action Owner context-ს.

15. Risk Acceptance

15.1 გამოყენების პრინციპი

Risk Acceptance არის formal response და გამოიყენება მხოლოდ მაშინ, როდესაც mitigation/avoid/transfer გზები შეფასებულია და management იღებს მიმდინარე exposure-ს.

15.2 ფორმის ველები

- Acceptance rationale;
- Initiator;
- Approver;
- Approval date;
- Valid until;
- Review date.

15.3 Validation

- responseType = Accept -> rationale required;

- status = Accepted -> approver, approvalDate და validUntil required.

15.4 Six-month validity

Framework-ის მიხედვით acceptance მოქმედებს approval date-დან ექვსი თვის განმავლობაში და expiration-ზე უნდა გადაიხედოს.

Current Phase 1:

- default date ექვს თვეს იყენებს;
- user-ს შეუძლია validUntil ხელით შეცვლა;
- approvalDate-ის ცვლილებისას six-month formula ავტომატურად არ enforce-დება.

Phase 2 requirement: API validation-მა უნდა უზრუნველყოს validity policy, approval authority, expiry reminders, re-approval და immutable decision history.

15.5 High/Significant acceptance

High ან Significant risk-ის acceptance exceptional decision-ად უნდა ჩაითვალოს და მოითხოვოს შესაბამისი approval/escalation path. მიმდინარე local prototype-ში hierarchical approval workflow არ არის.

16. Individual Risk View

16.1 Header band

Header აჩვენებს:

- Risk Reference და title;
- Category/Business Unit;
- Risk Owner;
- Action Owner context;
- Inherent/Residual/Target scores;
- Target Date;
- current status.

16.2 Tabs

1. **Overview**
2. **Assessment**
3. **Controls**
4. **Actions**
5. **Trend & Audit**

16.3 Overview

Overview მოიცავს:

- Cause/Event/Consequence cards;
- risk status narrative;
- response, review date და risk type;
- Historical Trend;
- Direction to Target;

- 12-month Outlook;
- Existing Controls summary;
- Remediation Actions table;
- acceptance summary, თუ response = Accept.

16.4 Assessment tab

- სამი score card;
- სამი 5x5 matrix;
- არჩეული cells;
- impact/likelihood guidance;
- assessment history table;
- residual trend chart.

16.5 Controls tab

ყველა structured control-ის read view, owner/effectiveness/type/frequency/evidence context-ით.

16.6 Actions tab

ყველა remediation action-ის დეტალური view progress/status/deadline/owner-ით.

16.7 Trend & Audit tab

- assessment history chronological information;
- risk-specific audit events;
- actor, timestamp, summary და change labels.

17. Trend, Direction და Outlook Logic

17.1 Historical Trend

History array-ის ბოლო ორ assessment snapshot-ს ადარებს Residual numeric score-ით.

history.length < 2	-> New
current residual < previous	-> Improving
current residual > previous	-> Worsening
current residual == previous	-> Stable

17.2 Direction to Target

Target score < Residual score	-> Decreasing toward target
Target score = Residual score	-> At target / Stable
Target score > Residual score	-> Increasing

17.3 Risk Outlook

Risk Outlook არის ხელით არჩეული expected direction მომდევნო 12 თვისთვის:

- Increasing;
- Stable;
- Decreasing.

Outlook არ უნდა გადაიწეროს ავტომატური trend calculation-ით, რადგან ის management judgement-ს ასახავს.

17.4 Reporting usage

Dashboard/Reports შეიძლება გამოიყენოს:

- trend distribution;
- outlook distribution;
- top improving/worsening risks;
- filter by outlook.

18. Assessment History და Audit Trail

18.1 History vs Audit

Mechanism	დანიშნულება
Assessment History	score state-ის snapshot; trend analysis
Audit Event	ვინ, როდის, რა entity-ზე რა ცვლილება შეასრულა

18.2 Current audit model

Current event ინახავს:

- ID;
- timestamp;
- actor ID;
- action code;
- entity type და ID;
- summary;
- optional change labels.

18.3 მიმდინარე შეზღუდვები

Phase 1 audit:

- localStorage state-ის ნაწილი;
- user-ს Developer Tools-ით შეუძლია შეცვალოს;
- before/after JSON სრულად არ ინახება;
- hash-chain/immutable storage არ აქვს;
- delete protection არ აქვს.

18.4 Phase 2 audit requirement

Backend event-ში რეკომენდებულია:

actor user snapshot
organization / business unit
entity type and ID
risk / assessment / control / action links
before JSON
and after JSON
changed fields
reason
request/session/IP/user-agent
occurredAt
previousHash + eventHash

Risk update და audit insert ერთ database transaction-ში უნდა შესრულდეს.

19. Dashboard Builder

19.1 მიზანი

Dashboard Builder ქმნის reusable management dashboards-ს live Risk Register dataset-იდან.

19.2 Dashboard filters

- Business Unit;
- Risk Category;
- Risk Status;
- Residual Rating;
- Risk Type;
- Risk Owner;
- Risk Outlook.

Filters dashboard definition-ში ინახება და გახსნისას აღდგება.

19.3 Dashboard CRUD

- Add dashboard;
- Edit metadata;
- Duplicate;
- Delete;
- Shared toggle;
- accent color;
- English/Georgian name and description.

Rules:

- ბოლო დარჩენილი dashboard ვერ წაიშლება;
- dashboard delete-ისას Report Templates-ში მასზე მიბმული dashboard sections იშლება, რათა broken reference არ დარჩეს;
- duplicate ქმნის ახალ ID-სა და დამოუკიდებელ widget collection-ს.

19.4 Widget types

Type	მთავარი options
Metric	Total Risks, Open Risks, Significant Residual, Overdue Actions, Emerging Risks, Completed Actions
Heatmap	Inherent/Residual/Target 5x5 distribution
Distribution/Bar	Rating, Category, BU, Status, Response, Outlook, Trend, Action Status
Top Risks	Inherent/Residual/Target; limit
Action Plan Progress	action status/progress summary
Recent Activity	audit events

Type	მთავარი options
Trend Summary	Improving/Worsening/Stable/New

19.5 Widget configuration

- English/Georgian title;
- foreground/accent color;
- background color;
- grid span: 3/4/6/8/12;
- score basis;
- grouping;
- metric type;
- limit 1-20;
- add/duplicate/remove/reorder.

19.6 Metric rules

Total Risks = all filtered risks
 Open Risks = status != Completed
 Significant Residual = residual rating == Significant
 Overdue Actions = due date passed and not Completed
 Emerging Risks = risk.type == Emerging
 Completed Actions = action.status == Completed

20. Report Template Builder

20.1 მიზანი

Reports module ქმნის reusable report templates-ს Dashboard-ების, open text-ისა და compact Risk Register sections-ის კომბინაციით.

20.2 Template CRUD

- Create;
- Edit;
- Duplicate;
- Delete;
- English/Georgian name and description;
- Print/PDF via browser print.

20.3 Section types

Dashboard section

- ირჩევს saved dashboard-ს;
- აქვს დამოუკიდებელი BU/category/status/rating/type/owner/outlook filters;
- report-ში შეიძლება რამდენიმე dashboard section იყოს სხვადასხვა scope-ით.

Open Text section

- English/Georgian title;
- English/Georgian narrative;

- გამოიყენება Executive Summary, Management Commentary, Recommendations და Decisions Required ტექსტებისთვის.

Compact Risk Register section

- selectable base columns;
- selectable active custom-attribute columns;
- independent filters;
- columns order template configuration-ით;
- მინიმუმ ერთი selected column უნდა დარჩეს.

20.4 Section lifecycle

- Add;
- Move up/down;
- Duplicate;
- Delete;
- configure independently.

20.5 Print/PDF

Phase 1-ში PDF export browser print dialog-ზეა დამოკიდებული. Phase 2-ში რეკომენდებულია server-side report rendering versioned template snapshot-ით.

21. Bilingual UI, navigation და accessibility

21.1 i18n

- Runtime toggle: English / ქართული;
- master data-ს აქვს English და Georgian labels;
- თუ ქართული თარგმანი ცარიელია, UI fallback-ად English value-ს იყენებს;
- user-entered risk narratives შეიძლება ერთ ენაზე დარჩეს; automatic translation არ ხდება.

21.2 Navigation

Risk routes იყენებს smooth enter transition-ს და scroll-to-top behavior-ს. prefers-reduced-motion შემთხვევაში animation მცირდება/ითიშება.

21.3 Accessibility baseline

- interactive control-ს უნდა ჰქონდეს visible label ან aria-label/title;
- keyboard focus visible უნდა იყოს;
- color-only meaning არ არის საკმარისი - rating/status ტექსტიც ჩანს;
- dialogs უნდა იხურებოდეს Cancel/Close-ით და focus არ უნდა დაიკარგოს.

22. Persistence და Repository Logic

22.1 Storage key

erm-risk-management-v3-state

22.2 Current repository

```
AppRepository
├── LocalStorageRepository <- active Phase 1
└── ApiRepository         <- Phase 2 scaffold
```

UI component არ უნდა იძახებდეს პირდაპირ localStorage-ს. Update pattern:

```
read AppState
-> deep clone
-> apply mutator
-> repository.save
-> refresh context/query
```

22.3 Schema version

Current schema: 8.

Migration layer:

- ამატებს missing default roles/users/site content;
- აღადგენს missing dashboard/report defaults;
- ახდენს old saved-view shape-ის შევსებას;
- repair-ს უკეთებს Business Unit invalid parent/cycle-ს;
- აშორებს invalid BU references user scopes-იდან;
- ინარჩუნებს risks, controls, actions, history და audit-ს.

22.4 Backup

Risk data სრულ backup-ში შედის Administration -> Data Tools -> Export JSON-ით. სხვა browser/computer-ზე გადასატანად საჭიროა Export -> Import.

23. Error Handling და Edge Cases

Scenario	Expected behavior
User opens hidden risk ID	Access denied / not found state
No categories/BUS/users	New Risk Save იბლოკება შესაბამისი validation-ით
Matrix cell missing/corrupt	migration/default matrix repair; fallback rating required
Removed category still used by old risk	category უნდა გახდეს inactive, record reference უნდა დარჩეს
Due date passed	overdue visual state computed
Empty control/action title	item removed on Save
Import invalid JSON	state არ იცვლება; error message
Import valid but old schema	migrate/repair before persist
Delete last dashboard	blocked
Report references deleted dashboard	related sections removed on dashboard deletion
Default Saved View deleted	default cleared; Register remains usable
Browser storage cleared	data lost unless JSON backup exists

24. Phase 2 Backend Requirements

24.1 Server-side boundaries

Backend უნდა გახდეს authoritative source შემდეგზე:

- authentication/session;
- RBAC და record-level authorization;
- matrix calculation/versioning;
- validation;
- workflow status transitions;
- acceptance approvals;
- audit ledger;
- reminders/notifications;
- attachment/evidence storage;
- concurrency control;
- backup/restore.

24.2 Recommended entities

organizations
users
memberships
roles
permissions
business_units
categories
custom_attributes
risks
risk_owner_assignments
risk_assessments
assessment_history
controls
control_versions
remediation_actions
acceptances
approvals
attachments
dashboards
report_templates
saved_views
audit_events

24.3 API behavior

- optimistic version field on Risk/Assessment/Action;
- 409 Conflict stale update-ზე;
- score/rating calculated server-side;
- approved assessment immutable, new version required;
- owner assignment history retained;
- soft-delete master data/users;
- export endpoints authorized and audited;
- all mutation endpoints write audit within same transaction.

25. QA Acceptance Criteria - Risk Management

25.1 Authentication and scope

- ყველა demo role შედის სწორი credential-ით;
- invalid credential rejected;
- each role sees only expected risks;
- Auditor cannot trigger mutation;
- direct route access obeys visibility rules.

25.2 Risk Register

- search covers all documented fields;
- each filter works independently and combined;
- BU filter includes descendants only;
- sorting toggles direction;
- Compact/Detailed persists through Saved View;
- columns persist through Saved View;
- default view auto-applies;
- remove view works;
- CSV/Excel contains filtered data.

25.3 Risk CRUD

- required validation works;
- generated reference correct;
- Cancel does not persist;
- score change adds history;
- non-score change does not add false history;
- audit event created;
- owner-specific partial update cannot alter protected data.

25.4 Assessment

- all 25 cells resolve configured rating;
- score = Impact x Likelihood;
- matrix reset restores 2026 defaults;
- three score types stay independent;
- trend logic matches history.

25.5 Controls and Actions

- quick control input creates one control per unique line;
- control owner only edits own controls;
- action owner only edits own actions;
- overdue rule works;
- progress values remain 0-100;
- Overview summary displays required columns.

25.6 Dashboard and Reports

- dashboard CRUD and last-dashboard guard;
 - all 7 widget types render;
 - filters saved/restored;
 - color/span/reorder/duplicate persist;
 - report sections add/move/duplicate/delete;
 - compact register cannot have zero columns;
 - Print/PDF view opens cleanly.
-

26. წყაროები და ინტერპრეტაციის წესი

- **Enterprise Risk Management Framework v1.0 Final:** lifecycle, governance, owner responsibilities, controls, actions, acceptance, monitoring/reporting.
- **Risk Rating Matrix 2026:** 12-month likelihood, Impact criteria, exact 5x5 rating matrix and guidance.
- **LEG-POL-001 - რისკების მართვის პოლიტიკა:** ქართული ტერმინოლოგია, Cause/Event/Consequence, register field descriptions, trend/outlook distinction and risk identifier concept.
- **Request for the change:** Saved Views, equal risk chips, OU-style Business Unit scope.
- **Request for change 2:** Dashboard Builder, Report Template Builder, detailed register and risk overview improvements.
- **AIZEN v4.1.0 as-built implementation:** current Phase 1 UI/state behavior.

Conflict rule: current assessment engine uses Risk Rating Matrix 2026-ის 12-month likelihood. LEG-POL-001-ის ძველი 3-year likelihood გამოიყენება მხოლოდ historical/translation context-ად და არ ცვლის 2026 calculation-ს.

AIZEN Risk Administration Module - ფუნქციური და ლოგიკური

სპეციფიკაცია

დოკუმენტის ტიპი: Administration Functional Specification / Configuration Manual

პროდუქტის ვერსია: AIZEN Risk & Compliance 4.1.0

დოკუმენტის ვერსია: 1.0

სტატუსი: As-built Phase 1 specification with Phase 2 requirements

მიზნობრივი აუდიტორია: Super Administrator, Administrator, Product Owner, Risk Team, Frontend, Backend, QA, Information Security

1. დოკუმენტის მიზანი

ეს დოკუმენტი აღწერს Risk Administration მოდულის ყველა ფუნქციას, ბიზნეს-წესსა და მონაცემთა დამოკიდებულებას. მიზანია ადმინისტრატორმა და development/QA გუნდმა ერთნაირად გაიგოს:

- რომელი master data მართავს Risk Management მოდულს;
- როგორ მუშაობს Categories, Business Unit hierarchy და dynamic Custom Attributes;
- როგორ იქმნება user და როგორ ენიჭება roles/scopes;
- როგორ ითვლება module permission;
- როგორ კონფიგურირდება 5x5 Risk Rating Matrix;
- რა განსხვავებაა client branding-სა და AIZEN public-site CMS-ს შორის;
- რატომ არის SSO მიმდინარე ეტაპზე roadmap configuration და არა რეალური login;
- როგორ მუშაობს backup/import/reset და schema repair;
- რა უნდა გახდეს server-side enforced Phase 2-ში.

2. Access Boundary

2.1 ვის აქვს Administration-ზე წვდომა

Risk Administration route-ზე წვდომა აქვს მხოლოდ:

- **Super Administrator;**
- **Administrator.**

Risk Manager, Risk Owner, Control Owner, Action Owner და Auditor ვერ ხედავენ sidebar entry-ს და direct route access-ისას იღებენ Access Denied/redirect behavior-ს.

2.2 Website Administration-ისგან განსხვავება

არსებობს ორი განსხვავებული ადმინისტრაციული სივრცე:

სივრცე	დანიშნულება	წვდომა
Risk Administration	risk categories, BUs, users, roles, matrix, branding, SSO roadmap, data tools	Administrator + Super Administrator
Website Administration	public website, About Us, solutions, team members, demo media	მხოლოდ Super Administrator

Website Administration არ არის ჩვეულებრივი module permission matrix-ის ნაწილი; იგი დაცულია dedicated Super Admin role guard-ით.

2.3 Phase 1 security note

Administration guard client-side-ია. ტექნიკურად knowledgeable user-ს შეუძლია browser state-ის შეცვლა. ამიტომ Phase 1 access არის workflow simulation, არა production security boundary.

3. Administration Navigation

Administration მოდული შეიცავს ათ განყოფილებას:

1. Administration Overview;
2. Categories;
3. Business Units;
4. Custom Attributes;
5. Users;
6. Roles & Permissions;
7. Rating Matrix;
8. Branding;
9. SSO / SAML Roadmap;
10. Data Tools.

Sidebar selection URL/state-ის მიხედვით ცვლის workspace-ს. სხვა module-ზე გადასვლისას unsaved modal draft არ persist-დება.

4. Administration Overview

4.1 მიზანი

Overview არის კონფიგურაციის health summary და სწრაფი navigation point.

4.2 Metrics

Overview ითვლის:

- Active Categories;
- Active Business Units;
- Active Users;
- Active Custom Attributes;
- Roles;
- Audit Events.

4.3 Calculation rules

```
Active item count = items where active/status == true/Active
Role count       = all configured roles
Audit count      = current AppState.auditEvents.length
```

4.4 Limitations

Overview არ ამოწმებს configuration completeness-ს. მაგალითად, შეიძლება active category არსებობდეს Georgian label-ის გარეშე ან user ჰქონდეს conflicting roles. Phase 2-ში სასურველია configuration health checks.

5. Risk Categories

5.1 მიზანი

Risk Category master data გამოიყენება:

- risk identification/classification-ში;
- Risk Register filters-ში;
- Dashboard/Report grouping/filtering-ში;
- data export-ში;
- bilingual labels-ში.

5.2 Hierarchy

Category-ს აქვს ორი დონე:

- **Level 1** - broad group, მაგალითად Strategic, Operational, Financial;
- **Level 2** - specific risk category.

Risk record იწახავს category ID-ს და UI category object-იდან იღებს Level 1/Level 2 labels-ს.

5.3 Fields

Field	Required	აღწერა
id	system	technical ID
level1En	yes	Level 1 English
level1Ka	optional	Level 1 Georgian
level2En	yes	Level 2 English
level2Ka	optional	Level 2 Georgian
active	yes	selectable status

5.4 Add/Edit workflow

1. Administrator clicks Add Category or Edit.
2. Modal opens with cloned values.
3. English required fields are validated.
4. On Save, state is updated and audit event appended.
5. Risk dropdowns and filters refresh from new state.

5.5 Activate/Deactivate

Used category should not be hard-deleted. Inactive behavior:

- აღარ ჩანს ახალი risk-ის normal selection list-ში;
- ძველი risk ინარჩუნებს category reference-ს;
- historical reporting can still resolve the label;

- Administrator can reactivate it.

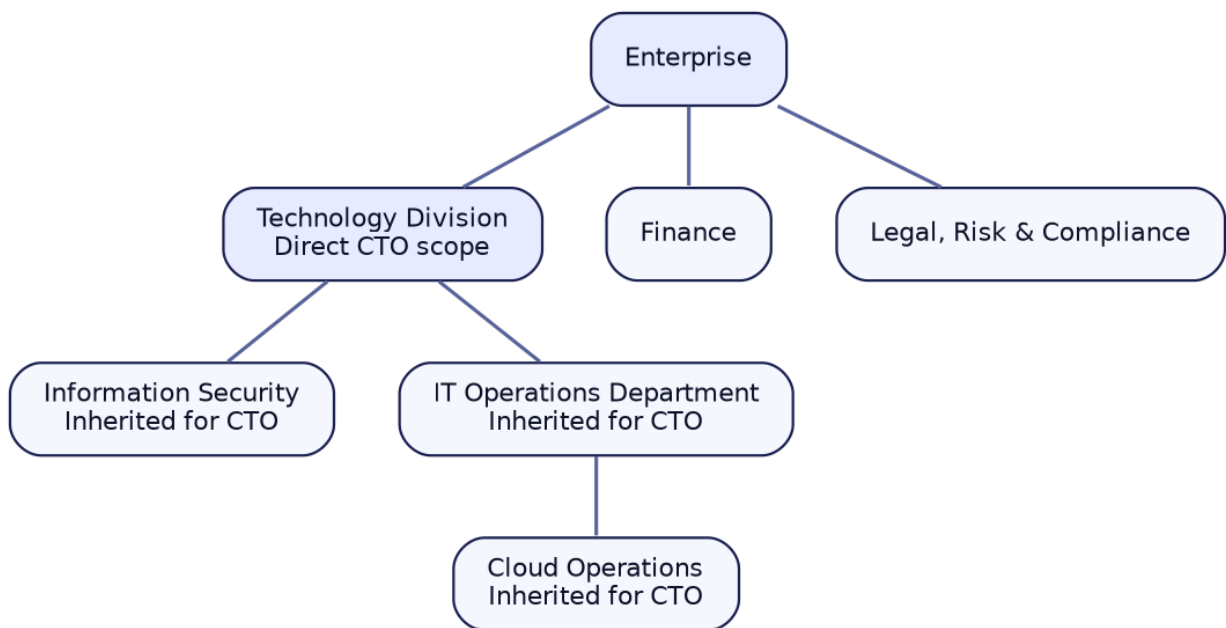
5.6 Current limitations

- duplicate Level 2 name prevention is not strongly enforced;
- category merge/re-map workflow does not exist;
- category version history is audit summary only.

5.7 Phase 2 requirements

- organization-scoped uniqueness policy;
- soft-delete with validFrom/validTo;
- bulk import/export;
- category usage count and safe remapping;
- API-side validation.

6. Business Units - OU-style Hierarchy



Business Unit inheritance example

6.1 მიზანი

Business Unit hierarchy ასრულებს ორ ფუნქციას:

1. risk ownership/reporting structure;
2. record-level access scope.

6.2 Tree model

```

Enterprise
├── Finance
├── Legal, Risk & Compliance
├── Technology Division
│   ├── Information Security
│   └── IT Operations Department
  
```

თითო node-ს შვიდობა ჰყავდეს შეუზღუდავი descendants.

6.3 Fields

Field	Required	აღწერა
id	system	technical ID
code	yes	unique uppercase business code
nameEn	yes	English name
nameKa	optional	Georgian translation
parentId	optional	parent node; blank = root
active	yes	selectable/effective status

6.4 Tree UI

- expand/collapse per node;
- add root;
- add child directly under selected parent;
- edit node;
- active/inactive toggle;
- hierarchy path display;
- direct users count;
- directly scoped risk count;
- descendant count.

6.5 Parent validation

Save იბლოკება, თუ:

- node საკუთარ parent-ად არის არჩეული;
- selected parent current node-ის descendant-ია და cycle წარმოიქმნება;
- Business Unit code duplicate-ია;
- required name/code ცარიელია.

6.6 Effective scope inheritance

თუ user-ს პირდაპირ ენიჭება parent unit, effective scope მოიცავს parent-ს და ყველა descendant-ს.

Direct scope: Technology Division

Effective scope:

- Technology Division
- Information Security
- IT Operations Department
- any future child under Technology Division

Child unit assignment არ იძლევა parent/sibling access-ს.

6.7 Direct vs inherited display

User editor-ში:

- direct scope არის selectable checkbox;
- inherited descendant მონიშნულია Inherited მდგომარეობით;
- inherited unit ცალკე ვერ მოიხსნება, სანამ parent direct scope აქტიურია;
- direct და effective scope counts ცალ-ცალკე ჩანს.

6.8 Risk relation

Risk ინახავს ერთ businessUnitId-ს. Business Unit filter parent node-ზე არჩევისას ითვლის descendant risks-საც. Risk reference prefix ახალი risk-ის შექმნისას არჩეული unit code-დან გენერირდება.

6.9 Deactivation

Business Unit deactivation არ უნდა წაშალოს historical references. ახალი risk/user assignment-ში inactive node აღარ უნდა იყოს default option, მაგრამ ძველი records უნდა დარჩეს readable.

6.10 Import/migration repair

Schema repair:

- invalid parentId -> root/valid default;
- self-parent -> parent cleared;
- detected cycle -> broken link cleared;
- user scopes-იდან non-existing BU IDs იშლება;
- old flat BUs იღებს parent structure defaults-ს.

6.11 Phase 2 requirements

- recursive CTE/closure table or materialized path;
- effective scope query server-side;
- unit move impact preview;
- approval/audit for hierarchy changes;
- risk/user count before deactivation;
- no orphan records.

7. Custom Attributes - Dynamic Risk Schema

7.1 მიზანი

Custom Attributes საშუალებას აძლევს organization-ს დაამატოს Risk Register fields UI code rewrite-ის გარეშე.

7.2 Supported types

- Text;
- Number;
- Date;
- Select;
- User picker.

7.3 Fields

Field	აღწერა
id	stable key; risk.custom object-ის key
labelEn	English label
labelKa	Georgian label

Field	აღწერა
type	text/number/date/select/user
options	select values, comma-separated configuration
active	editor/report availability
showInRegister	Risk Register selectable column

7.4 Runtime behavior

Active attribute:

- ჩანს Risk Editor -> Custom Fields-ში;
- შეიძლება გამოჩნდეს Risk Register-ში, თუ showInRegister = true;
- ემატება Compact Report Register-ის column options-ში;
- შედის full export-ში;
- ინახება risk.custom[attributeId] ფორმატით.

7.5 Defaults

- Number -> 0;
- other types -> empty string.

7.6 Select options

Administrator წერს comma-separated values-ს. UI trim-ს უკეთებს თითო value-ს და ცარიელ values-ს გამორიცხავს.

7.7 Deactivation

Inactive attribute:

- აღარ ჩანს ახალი edit/report column configuration-ში;
- risk.custom-ში არსებული value არ იშლება;
- reactivation-ისას ძველი values ისევ ხელმისაწვდომია.

7.8 Current limitations

- required flag არ არსებობს;
- validation rules/min/max/regex არ არსებობს;
- option rename migration არ არსებობს;
- type change შეიძლება ძველ values-ს შეუთავსებელი გახადოს;
- field-level permissions არ არსებობს.

7.9 Phase 2 requirements

- versioned attribute definitions;
- typed server validation;
- required/default rules;
- option dictionary IDs instead of label-only values;
- field-level access and audit;
- safe type-change migration.

8. Users

8.1 მიზანი

User Directory გამოიყენება login-ის, ownership pickers-ის, role permissions-ისა და Business Unit scope-ისთვის.

8.2 Fields

Field	Required	აღწერა
id	system	technical ID
name	yes	display name
title	optional	position/title
email	yes	login identifier
password	Phase 1 yes	demo local password
status	yes	Active/Inactive
roleIds	yes	one or more roles
businessUnitIds	yes	one or more direct scopes

8.3 Add/Edit workflow

1. Open User Editor.
2. Enter name/email/password/title/status.
3. Select one or more roles.
4. Select direct Business Unit scopes.
5. Review effective inherited scope.
6. Save -> state update + audit event.

8.4 Validation

Save იბლოკება, თუ:

- name ცარიელია;
- email ცარიელია;
- role არ არის არჩეული;
- Business Unit scope არ არის არჩეული.

8.5 Active/Inactive

Inactive user:

- ვერ შედის local login-ით;
- არ უნდა გამოჩნდეს owner candidate list-ში;
- historical owner/audit references უნდა დარჩეს readable;
- reactivation შესაძლებელია.

8.6 User deletion policy

Hard delete არ არის რეკომენდებული, რადგან user შეიძლება დაკავშირებული იყოს risk ownership-ს, controls-ს, actions-ს, assessments-სა და audit events-ს. გამოიყენება Inactive status.

8.7 Multiple roles

Module permission არის ყველა role permission-ის maximum. Record visibility current Phase 1-ში named role precedence-ს იყენებს; Phase 2-ში scopes union-ად უნდა გაერთიანდეს.

8.8 Current limitations

- duplicate email uniqueness მკაცრად არ მოწმდება;
- passwords plain text-ად ინახება;
- password complexity/expiry/MFA არ არსებობს;
- failed-login lockout არ არსებობს;
- user invitation/reset flow არ არსებობს;
- ownership reassignment wizard არ არსებობს.

8.9 Phase 2 requirements

- SSO/local identity provider integration;
- unique normalized email;
- MFA/session controls;
- invitation/deactivation workflow;
- bulk provisioning;
- owner reassignment before deactivation;
- actor snapshots in audit.

9. Roles & Permissions

9.1 Permission modules

Matrix მოიცავს:

- Dashboard;
- Register;
- Risks;
- Controls;
- Actions;
- Reports;
- Audit;
- Administration.

9.2 Permission levels

- None;
- Read;
- Edit.

9.3 Default roles

Role	Dashboard	Register	Risks	Controls	Actions	Reports	Audit	Administration
Super Administrator	Edit	Edit	Edit	Edit	Edit	Edit	Edit	Edit
Administrator	Edit	Edit	Edit	Edit	Edit	Edit	Edit	Edit

Role	Dashboard	Register	Risks	Controls	Actions	Reports	Audit	Administration
Risk Manager	Read	Edit	Edit	Edit	Edit	Edit	Read	None
Risk Owner	Read	Read	Edit	Edit	Edit	Read	Read	None
Control Owner	Read	Read	Read	Edit	Read	None	Read	None
Action Owner	Read	Read	Read	Read	Edit	None	Read	None
Auditor	Read	Read	Read	Read	Read	Read	Read	None

9.4 Permission aggregation

მრავალ role-ზე თითო module-ის effective permission არის ყველაზე მაღალი:

```
max(role.permissions[module])
```

9.5 New role

Administrator-ს შეუძლია custom role-ის შექმნა:

- English name;
- Georgian name;
- description;
- module permission matrix.

9.6 System role behavior

Roles-ს აქვს system/custom metadata, მაგრამ current Phase 1 system roles სრულად immutable არ არის. Administrator-ს შეუძლია მათი permission-ის შეცვლა.

Phase 2 recommendation: Super Administrator და core security roles protected უნდა იყოს, privilege escalation-sensitive change კი dual approval/audit-ს მოითხოვდეს.

9.7 Module permission vs record permission

risks: edit მარტო არ ნიშნავს ყველა risk-ის edit-ს. საბოლოო გადაწყვეტილება არის:

```
Module Permission
AND Route Permission
AND Record Visibility
AND Record-specific Edit Rule
```

9.8 Delete behavior

Current UI-ში role delete workflow არ არის. Role deactivation/versioning სასურველია, რადგან role შეიძლება user-ზე იყოს მინიჭებული.

10. Risk Rating Matrix Configuration

10.1 მიზანი

Matrix editor განსაზღვრავს თითო Impact/Likelihood intersection cell-ის rating-ს და rating colors-ს.

10.2 25-cell model

Matrix არის 5 Impact x 5 Likelihood. თითო cell იწახვს:

```
impact
likelihood
rating = Low | Medium | High | Significant
```

10.3 Color configuration

Administrator ცვლის:

- Low color;
- Medium color;
- High color;
- Significant color.

ცვლილება მაშინვე აისახება:

- Risk Register chips;
- Risk Detail cards/matrices;
- Dashboard heatmap/distributions;
- Reports;
- rating filters.

10.4 Restore defaults

Restore 2026 defaults აბრუნებს:

Impact Likelihood	1	2	3	4	5
5	Medium	High	High	Significant	Significant
4	Low	Medium	High	High	Significant
3	Low	Medium	Medium	High	High
2	Low	Low	Medium	Medium	High
1	Low	Low	Low	Low	Medium

Colors:

Low #00B050
Medium #FFF200
High #FFB900
Significant #F32121

10.5 Current limitations

- impact/likelihood labels და probability ranges UI-დან არ რედაქტირდება;
- matrix version history არ არსებობს;
- historical assessment runtime-ზე current matrix-ს იყენებს;
- approval workflow matrix change-ზე არ არსებობს.

10.6 Phase 2 requirements

- versioned matrices;
- effective date;
- draft/approved status;
- historical assessment pinned to matrix version;
- change justification and approver;
- backend-side calculation;
- validation that all 25 cells exist.

11. Branding

11.1 Scope

Risk Administration Branding section მართავს **client company logo**-ს, რომელიც application header-ის top-right-ში ჩანს.

11.2 Upload logic

- image file browser-ში იკითხება;
- ინახება base64/data URL-ად AppState-ში;
- refresh-ის შემდეგ localStorage-დან აღდგება;
- remove action აბრუნებს placeholder/default state-ს.

11.3 AIZEN brand separation

AIZEN cotton-flower logo და public-site content Website Administration/Super Admin scope-ია. Risk Administrator-ს client logo-ის შეცვლა შეუძლია, მაგრამ AIZEN public website-ის ტექსტებსა და team members-ს ვერ ცვლის.

11.4 Limitations

- image size/type hard limit შეზღუდულია browser capabilities-ით;
- base64 ზრდის localStorage size-ს;
- no malware scanning;
- no central asset versioning.

11.5 Phase 2

- S3/object storage;
- MIME/size validation;
- image optimization;
- signed upload;
- organization branding version history.

12. SSO / SAML Roadmap Configuration

12.1 მიმდინარე სტატუსი

ეს section ინახავს მომავალ identity integration-ის კონფიგურაციის draft-ს. იგი არ ასრულებს რეალურ SAML authentication-ს Phase 1-ში.

12.2 Fields

- Provider Name;
- IdP Entity ID;
- Metadata URL/XML reference;
- ACS URL;
- Email attribute;
- Role/Group attribute;
- role mappings;
- Enabled toggle.

12.3 Intended future flow

User opens Sign In
-> redirect to IdP
-> IdP authenticates
-> signed SAML assertion posted to ACS
-> backend validates signature/audience/time
-> extract email/groups
-> match pre-provisioned Active user
-> map groups to AIZEN roles
-> create secure session

12.4 Pre-provisioning rule

Product requirement არის, რომ SAML user AIZEN user directory-ში წინასწარ არსებობდეს და იყოს Active. JIT provisioning default-ად გამორთული უნდა იყოს.

12.5 Current audit behavior

Enabled toggle ქმნის explicit audit event-ს. Text fields current local implementation-ში state-ში იწახება, მაგრამ ყველა field change-ზე granular audit event არ იქმნება.

12.6 Phase 2 mandatory controls

- signed assertion validation;
- metadata/certificate rotation;
- replay prevention;
- audience/recipient/time validation;
- HTTPS-only ACS;
- secure HttpOnly session;
- domain allowlist;
- pre-provisioned user check;
- group-to-role mapping validation;
- login success/failure audit;
- break-glass admin account.

13. Data Tools

13.1 Export Full JSON Backup

Export serializes complete AppState, მათ შორის:

- schema version;
- users/roles;
- categories/BUs/attributes;
- risks/controls/actions/history/audit;
- dashboards/reports/saved views;
- branding;
- SSO draft;
- public-site content.

13.2 Import JSON Backup

Import flow:

1. user selects JSON file;
2. parse JSON;
3. basic structural validation - minimum risks/users arrays;
4. run migration/repair;
5. replace current state;
6. persist to repository;
7. refresh app.

Invalid JSON ან invalid structure state-ს არ ცვლის.

13.3 Reset Demo Data

- confirmation required;
- replaces current state with seed state;
- all local user-entered data is lost;
- schema/current defaults restored.

13.4 Export Audit JSON

Exports audit event array separately, typically for review or troubleshooting.

13.5 Operational warning

Data ინახება კონკრეტულ browser profile/path-ში. Browser storage clear, private browsing, file-path change ან device change შეიძლება data loss-ს გამოიწვიოს. Regular JSON backup სავალდებულო operational practice-ია Phase 1-ში.

13.6 Phase 2

- database backups/PITR;
- organization-scoped export;
- encrypted archive;
- import dry-run/validation report;
- audit of export/import/reset;
- privileged approval for destructive reset;
- retention policy.

14. Administration Audit Events

14.1 Expected audited actions

- category created/updated/status changed;
- Business Unit created/updated/status changed/moved;
- custom attribute created/updated/status changed;
- user created/updated/activated/deactivated/role or scope changed;
- role created/updated/permission changed;
- matrix cell/color reset/changed;

- branding changed;
- SSO enabled/disabled;
- data import/reset;
- full export/audit export - Phase 2 requirement.

14.2 Current audit quality

Current Phase 1 audit stores summary events, not immutable before/after versions. Some configuration fields may save without granular per-field event.

14.3 Phase 2

Administration changes are high-risk privileged actions. Backend audit should include:

- actor identity snapshot;
- before/after;
- changed fields;
- reason/ticket;
- session/request/IP;
- approval where required;
- tamper-evident retention.

15. Validation, Error Handling & Edge Cases

Scenario	Expected behavior
Non-admin opens admin route	Access denied/redirect
Category English labels empty	Save blocked
Used category deactivated	old risks retain reference
Duplicate BU code	Save blocked
BU parent creates cycle	Save blocked
Invalid BU hierarchy imported	migration repairs links
User has no role	Save blocked
User has no BU scope	Save blocked
User inactive	login/owner selection blocked
Role permission changed	effective access changes on next state evaluation
Matrix cell changed	all derived ratings recalculate
Last required matrix cell missing	migration/default repair required
Client logo too large	browser/localStorage may fail; show error recommended
Invalid JSON import	no state replacement
Reset clicked accidentally	confirmation required
System role edited to remove admin access	current build allows risk; Phase 2 must protect

16. Cross-module Dependencies

Administration object	გამოყენება სად
Category	Risk Editor, Register, Dashboard, Reports, Export
Business Unit	Risk ownership, user scope, access control, filters, reference generation
Custom Attribute	Risk Editor, Register columns, Reports, Export
User	Login, Risk/Control/Action Owner, audit actor
Role	routes, modules, edit/read controls
Rating Matrix	all score chips, matrices, filters, heatmaps, reports
Branding	application header/client identity
SSO Config	Phase 2 authentication
Data Tools	whole AppState lifecycle

Configuration change impact უნდა შეფასდეს cross-module regression-ით.

17. Recommended Change-Control Rules

მიუხედავად იმისა, რომ Phase 1-ში არ არის enforced, production administration-ისთვის რეკომენდებულია:

- risk matrix change requires approval and effective date;
- role permission change requires privileged review;
- Super Admin role cannot be deleted/deprivileged by ordinary admin;
- Business Unit move displays affected user/risk counts;
- user deactivation requires owner reassignment check;
- category/attribute deactivation shows usage count;
- destructive import/reset requires backup and typed confirmation;
- SSO certificate/metadata changes require connection test and rollback.

18. Phase 2 Data/API Requirements

18.1 Core endpoints

GET/POST/PATCH /categories
GET/POST/PATCH /business-units
GET/POST/PATCH /custom-attributes
GET/POST/PATCH /users
GET/POST/PATCH /roles
GET/PUT /rating-matrix
GET/PUT /branding
GET/PUT/TEST /identity-providers
POST /exports
POST /imports/validate
POST /imports/commit
GET /audit-events

18.2 Server-side rules

- tenant/organization isolation;

- unique normalized codes/emails;
 - soft delete;
 - optimistic versioning;
 - recursive BU scope;
 - permission change effective immediately/session refresh;
 - matrix version pinning;
 - all mutations transactionally audited;
 - admin endpoints deny non-privileged callers independent of UI.
-

19. QA Acceptance Criteria - Administration

19.1 Access

- Super Administrator and Administrator can open Risk Administration;
- all other roles cannot;
- Website Administration only Super Administrator can open.

19.2 Categories

- add/edit/deactivate/reactivate;
- bilingual labels;
- old risk keeps inactive category;
- audit event created.

19.3 Business Units

- add root and child;
- move node;
- expand/collapse;
- duplicate code rejected;
- self/descendant parent rejected;
- inherited scope correct;
- sibling/parent access not leaked;
- import repair works.

19.4 Custom Attributes

- all five types render;
- select options parse correctly;
- values persist on risk;
- showInRegister works;
- report column appears;
- deactivation hides field without deleting value.

19.5 Users

- required validation;
- multiple roles/scopes;
- inactive user cannot login;

- owner pickers exclude inactive user;
- effective scope count correct.

19.6 Roles

- custom role create/edit;
- None/Read/Edit effective behavior;
- highest permission across roles;
- route and buttons match permission;
- record-level scope still enforced.

19.7 Matrix

- every one of 25 cells editable;
- colors update all views;
- restore defaults exact;
- risk score remains Impact x Likelihood;
- matrix-driven filter/heatmap consistent.

19.8 Data Tools

- export valid JSON;
- import valid current and old schema;
- invalid import leaves state unchanged;
- reset confirmation and seed restore;
- audit export valid JSON.

20. წყაროები

- **Enterprise Risk Management Framework v1.0 Final:** governance roles, Risk Owner/Control Owner/Action Owner duties, lifecycle, monitoring/reporting.
- **Risk Rating Matrix 2026:** Impact/Likelihood definitions, rating matrix, colors/guidance baseline.
- **Risk Category Library 2026:** seed category structure.
- **LEG-POL-001:** ქართული terminology, risk register field meaning, identifier concept.
- **Request for the change:** OU-style Business Unit hierarchy and access inheritance; Saved View behavior.
- **Request for change 2:** Dashboard/Report administration requirements and register/detail presentation.
- **AIZEN v4.1.0 as-built code:** exact Phase 1 behavior described as Implemented/Current.

AIZEN Risk & Compliance - Cross-Module Architecture, RBAC, Data, Audit and Migration Logic

პროდუქტის ვერსია: 4.1.0
დოკუმენტის ვერსია: 1.0
სტატუსი: Phase 1 as-built + Phase 2 target architecture

1. მიზანი

ეს დოკუმენტი აღწერს Risk Management-სა და Risk Administration-ს შორის საერთო ტექნიკურ და ბიზნეს ლოგიკას:

- application state ownership;
- repository abstraction;
- authentication/session;
- module permissions;
- record-level scope;
- Business Unit inheritance;
- audit/history separation;
- localStorage persistence;
- schema migration and repair;
- import/export;
- error handling;
- backend migration contract.

2. Current Application Architecture

2.1 High-level layers



2.2 Responsibilities

Layer	პასუხისმგებლობა
Page/Component	display, local form draft, user interaction
AppDataContext	current AppState, update transaction, repository orchestration
Zustand	session/global UI state, not authoritative business data

Layer	პასუხისმგებლობა
Domain helpers	score, rating, trend, scope, formatters
Repository	read/save/reset/import/export persistence
LocalStorage	Phase 1 data store
ApiRepository	future REST adapter contract

2.3 State update pattern

Business data mutation უნდა გაიაროს ერთიანი path-ით:

1. Get current state
2. Deep-clone state
3. Apply deterministic mutator
4. Append audit/history where required
5. repository.save(nextState)
6. Publish next state to UI

Component-მა პირდაპირ localStorage.setItem არ უნდა გამოიყენოს.

3. AppState Boundaries

AppState მოიცავს:

```

schemaVersion
users
roles
categories
businessUnits
customAttributes
matrix
risks
savedViews
dashboards
reportTemplates
auditEvents
branding
ssoConfig
siteContent
session-related local references

```

3.1 ID references

Entities ერთმანეთს უკავშირდება stable string IDs-ით:

- risk.categoryId -> category.id;
- risk.businessUnitId -> businessUnit.id;
- risk.riskOwnerId -> user.id;
- control.ownerId -> user.id;
- action.ownerId -> user.id;
- user.roleIds -> role.id[];
- user.businessUnitIds -> businessUnit.id[];
- report dashboard section -> dashboard.id.

3.2 Derived values

არ უნდა იწახებოდეს, თუ deterministic-ად ითვლება:

- risk score;
- risk rating;
- historical trend;
- direction to target;
- effective Business Unit scope;
- overdue action indicator;
- filtered dashboard metrics.

Derived values runtime-ზე ითვლება authoritative inputs/configuration-იდან.

4. Authentication Logic - Phase 1

4.1 Login

1. email normalized comparison-ით ეძებს user-ს;
2. user status უნდა იყოს Active;
3. password plain-text equality-ით მოწმდება demo seed/state-ში;
4. success-ისას session user ID ინახება browser state/localStorage-ში;
5. refresh-ზე session აღდგება;
6. logout session reference-ს შლის.

4.2 Security limitation

Phase 1 login არ არის production authentication:

- password unhashed/plain text;
- no secure cookie;
- no MFA;
- no brute-force protection;
- local state editable;
- no server trust boundary.

4.3 Phase 2 target

- Microsoft Entra ID / Okta SAML/OIDC;
 - optional local break-glass accounts;
 - backend session with HttpOnly/Secure/SameSite cookie;
 - server-side active-user and role check on every request;
 - session revocation;
 - login success/failure audit;
 - MFA and conditional access delegated to IdP.
-

5. RBAC Evaluation

5.1 Permission scale

None = 0
Read = 1
Edit = 2

5.2 Effective module permission

For each module:

effective = max(permission from every assigned role)

5.3 Decision model

Mutation/read access საბოლოოდ განისაზღვრება ოთხი gate-ით:

Authenticated user
AND Module permission
AND Route/action permission
AND Record-level scope
AND Field-level role rule (where applicable)

5.4 Module list

- dashboard;
- register;
- risks;
- controls;
- actions;
- reports;
- audit;
- administration.

Website Administration separate Super Admin guard-ის.

6. Record-Level Scope

6.1 Named role rules

Role	Risk visibility
Super Administrator	all
Administrator	all
Risk Manager	all
Auditor	all, read-only
Risk Owner	assigned risks
Control Owner	risks with owned control
Action Owner	risks with owned action
Custom role	effective BU scope

6.2 Business Unit tree functions

Core helpers:

- get descendants;
- get ancestors;
- build full path;
- flatten tree for picker;

- calculate effective scope;
- detect/repair cycles.

6.3 Inheritance rule

Direct parent grant -> parent + all descendants
 Child grant -> child + descendants only
 No upward/sibling inheritance

6.4 Field-level partial update

Control Owner and Action Owner save does not trust hidden fields. Update code restores protected risk properties from original record and merges only authorized owned sub-records.

6.5 Phase 2 authorization model

Recommended database/API pattern:

```
module grant
+ organization membership
+ business unit closure/ancestor relation
+ explicit ownership relation
+ action-specific policy
```

Every endpoint must recompute access; client-supplied role/scope claims are not trusted.

7. Matrix and Risk Engine Contract

7.1 Inputs

```
assessmentType
impact 1..5
likelihood 1..5
matrixVersion/configuration
```

7.2 Outputs

```
score = impact * likelihood
rating = matrix[impact, likelihood]
color = ratingColors[rating]
```

7.3 Consistency requirement

Register chips, detail cards, heatmap, dashboard metrics, filters, reports and export must call the same risk engine. Duplicate rating logic in UI components is prohibited.

7.4 Backend migration

API should return or verify:

```
{
  "impact": 4,
  "likelihood": 5,
  "score": 20,
  "rating": "Significant",
  "matrixVersionId": "matrix-2026-v1"
}
```

Approved assessment should remain tied to its matrix version even after configuration changes.

8. History and Audit Separation

8.1 Assessment History

Purpose: time-series of score state.

Created when:

- new risk is created;
- inherent, residual or target object changes.

Not created for:

- narrative-only update;
- owner/status/control/action change without score change.

8.2 Audit Event

Purpose: actor/action trace.

Current fields:

id, timestamp, actorId, action, entityType, entityId, summary, changes

8.3 Global vs risk-specific audit

- global auditEvents supports Recent Activity/Administration review;
- risk-specific audit list supports Individual Risk timeline;
- event order newest first.

8.4 Phase 2 event taxonomy

risk.created
risk.updated
risk.status_changed
risk.owner_changed
assessment.created
assessment.submitted
assessment.approved
assessment.score_changed
control.created
control.updated
control.effectiveness_changed
remediation.created
remediation.status_changed
remediation.due_date_changed
acceptance.requested
acceptance.approved
acceptance.expired
user.created
user.status_changed
user.role_changed
role.permission_changed
business_unit.moved
matrix.updated
data.exported
data.imported

8.5 Transaction rule

```
BEGIN
  update business entity
  insert history/version if applicable
  insert audit event
COMMIT
```

If audit insert fails, mutation should roll back.

9. Persistence

9.1 Storage key

```
erm-risk-management-v3-state
```

The key is intentionally retained for backward compatibility.

9.2 Repository interface

```
interface AppRepository {  
  getState(): Promise<AppState>;  
  saveState(state: AppState): Promise<AppState>;  
  reset(): Promise<AppState>;  
  exportJson(state: AppState): string;  
  importJson(text: string): Promise<AppState>;  
}
```

9.3 LocalStorageRepository

Responsibilities:

- read JSON;
- seed when missing;
- migrate when old schema;
- repair invalid references;
- persist current schema;
- export/import/reset.

9.4 ApiRepository

Target behavior:

- same domain-facing interface;
 - Axios/HTTP implementation;
 - normalized server DTO mapping;
 - auth/session errors;
 - optimistic update/query invalidation;
 - no UI rewrite.
-

10. Schema Migration and Repair

10.1 Current schema

```
schemaVersion = 8
```

10.2 Migration principles

- migration is idempotent;
- never intentionally deletes valid risk data;
- missing new collections get defaults;
- invalid references are repaired or removed conservatively;
- current state saved only after successful migration.

10.3 Repair examples

Problem	Repair
Missing Super Admin role/user	add defaults
Old flat Business Units	add parentId/default tree
Self-parent/cycle	clear offending parent link
Missing parent	clear/replace invalid parent
User references deleted BU	remove invalid BU ID
Saved View lacks columns/mode	add default visibleColumns/viewMode
Missing dashboards/reports	restore seed defaults
Missing site content	add default public-site content
Matrix incomplete	merge/restore default cells/colors

10.4 Data-preservation guarantee

Migration should preserve:

- risk IDs/references;
- scores;
- controls/actions;
- ownership references when valid;
- assessment history;
- audit events;
- dashboard/report definitions;
- custom values.

10.5 Phase 2 migration

Local JSON import script should:

1. validate schema;
2. normalize IDs/emails/dates;
3. create organization;
4. import master data first;
5. import users/roles/scopes;
6. import risks and child entities;
7. import dashboards/reports/saved views;
8. store original source IDs for traceability;
9. produce reconciliation report;
10. archive original encrypted JSON.

11. Import / Export Logic

11.1 Full backup

Complete state export is intended for:

- browser/device transfer;
- manual backup;
- QA fixtures;
- Phase 2 migration source.

11.2 Risk Register export

CSV/XLS contains only current visible and filtered records. It is not a full backup and cannot restore application state.

11.3 Audit export

Audit-only JSON is review data, not restore package.

11.4 Phase 2 controls

- export permission separate from read permission;
- export event audited;
- data masking/column policy;
- large export job async;
- signed temporary download;
- import dry-run and approval.

12. State Concurrency

12.1 Phase 1

Single browser state uses last-save-wins. Multiple tabs/users can overwrite each other without conflict detection.

12.2 Phase 2

Every mutable aggregate should have version/ETag:

```
Client reads version 7
Client sends PATCH with If-Match: 7
Server updates -> version 8
Second stale client sends version 7 -> 409 Conflict
```

Risk, assessment, control and action may use separate version boundaries depending on collaboration requirements.

13. Error Handling

13.1 Categories

- validation error: inline field message;
- authorization error: deny + no state mutation;
- not found: safe empty/not-found view;
- import parse error: preserve current state;
- repository write failure: show error and keep modal draft;
- migration error: do not overwrite raw source; offer backup/export.

13.2 Phase 2 HTTP mapping

HTTP	Meaning
400	invalid data
401	unauthenticated
403	authenticated but not authorized
404	entity not found or hidden by scope
409	version/conflict/duplicate
422	business-rule validation
429	rate limit
500	unexpected server error

14. Security Boundaries

14.1 Phase 1 known risks

- localStorage data can be read/modified;
- passwords stored locally;
- audit tamperable;
- user can inspect all bundled seed data;
- file:// origin/path behavior differs by browser;
- no centralized backup/retention;
- no encryption at rest controlled by application.

14.2 Production controls

- HTTPS only;
- SSO/MFA;
- HttpOnly session;
- least privilege IAM;
- database encryption and backups;
- S3 evidence encryption/versioning;
- server-side RBAC/RLS;
- immutable/tamper-evident audit;
- secrets manager;
- CSP/security headers;
- dependency scanning;
- regular authorization tests.

15. Recommended Backend Architecture

15.1 Portable option

React SPA
-> NestJS REST API
-> PostgreSQL
-> S3/MinIO evidence
-> Redis/queue for reminders
-> SAML/OIDC adapter

15.2 AWS option

CloudFront + S3 static frontend
API Gateway / ALB
ECS Fargate NestJS API
RDS PostgreSQL
S3 attachments/audit archive
Cognito or direct Entra/Okta federation
CloudWatch + CloudTrail
Secrets Manager
SES notifications

15.3 Data-store rule

Transactional risk data should use PostgreSQL/RDS. S3 is appropriate for:

- evidence;
- logos/media;
- exports;
- snapshots;
- immutable audit archive.

S3 alone should not be the primary concurrent transactional risk database.

16. Suggested Database Relations

Organization 1--* Membership *--1 User
Membership *--* Role
Role 1--* Permission
BusinessUnit 1--* BusinessUnit(parent)
Membership *--* BusinessUnitScope
Category 1--* Risk
BusinessUnit 1--* Risk
User 1--* RiskOwnerAssignment
Risk 1--* Assessment
Risk 1--* Control
Risk 1--* RemediationAction
Risk 1--* Acceptance
Risk 1--* AuditEvent
Dashboard 1--* Widget
ReportTemplate 1--* ReportSection
User 1--* SavedView

17. API Contract Principles

- DTOs are explicit and versioned;
- client never sends calculated rating as authoritative;
- hidden fields cannot be mass-assigned;

- list endpoints apply scope before pagination;
- filter options are tenant/scope aware;
- soft-deleted referenced data remains resolvable for history;
- timestamps use UTC ISO 8601;
- dates without time use ISO date;
- IDs are opaque UUID/ULID;
- every mutation supports correlation/request ID.

18. Cross-Module Regression Matrix

Change	Must retest
Category update	Risk editor, register filter, dashboards, reports, export
BU move	user effective scope, risk visibility, filters, owner pickers, reference generation
Role permission	routes, buttons, API/record guards, auditor behavior
Matrix cell/color	all 3 assessments, chips, filters, heatmap, reports, history interpretation
Custom attribute	risk editor, register columns, saved views, report columns, export/import
User deactivation	login, owner pickers, historical display, assignments
Dashboard deletion	Report sections and print
Schema change	old backup import, current data persistence, reset seed

19. QA Baseline

Existing modular build QA baseline referenced by this documentation:

Area	Result
Architecture/build	58/58
Seed/domain	34/34
Public site	31/31
Authentication	32/32
RBAC	32/32
Super Admin CMS	15/15
Dashboard	12/12
Register/Risk	44/44
Reports	18/18
Administration	16/16
Migration/persistence/accessibility	13/13
Total	305/305

QA result indicates tested behavior for the referenced build; it does not remove Phase 1 security limitations or replace production penetration/performance testing.

20. Definition of Done for Backend Cutover

Backend cutover is complete only when:

- UI uses ApiRepository in production;
- all writes are server-validated;
- role/scope tests pass against API;
- SSO session works;
- matrix versioning works;
- risk owner/assessment/control/action history retained;
- immutable audit records created transactionally;
- local JSON migration reconciles counts;
- backup/PITR restore tested;
- evidence storage secured;
- performance and security tests pass;
- no production credential/data remains in localStorage.

AIZEN Functional Logic - Source Traceability Matrix

დოკუმენტის ვერსია: 1.0

მიზანი: ფუნქციური წესების წყაროს, ინტერპრეტაციისა და მიმდინარე იმპლემენტაციის დაკავშირება.

1. Source Priority

კონფლიქტის ან განსხვავებული ვერსიის შემთხვევაში გამოიყენება:

1. **Risk Rating Matrix 2026** - current Impact/Likelihood/rating calculation;
2. **Enterprise Risk Management Framework v1.0 Final** - lifecycle, governance, controls, actions, acceptance, monitoring/reporting;
3. **Risk Category Library 2026** - current category seed/master data;
4. **LEG-POL-001** - რისკების მართვის პოლიტიკა - ქართული ტერმინოლოგია და register field explanations;
5. **Change Requests** - product UX/administration/dashboard/report behavior;
6. **AIZEN 4.1.0 as-built code** - exact Phase 1 technical behavior.

Important interpretation

LEG-POL-001 (2022) likelihood table uses a 3-year horizon, ხოლო Risk Rating Matrix 2026 იყენებს next 12 months-ს. Current application calculation follows the 2026 matrix. Older Georgian policy is used for terminology/register structure, not as the active probability model.

2. Enterprise Risk Management Framework v1.0 Final

Source location	Source concept	AIZEN functionality
pp. 2-4	ERM principles, collaborative model, Three Lines, governance	role descriptions, governance-oriented reporting and access model
pp. 6-9	monitoring/reporting, Risk Appetite, KRI/RAG, lifecycle introduction	dashboards/reports context; future appetite/KRI roadmap
p. 9	four lifecycle stages: Identification, Assessment, Response, Monitoring/Reporting/Continuous Review	Risk workflow and documentation structure
p. 10	risk identification, current/emerging risk, register, owner/control/action owner	Risk Type, Risk Register, ownership fields
pp. 10-11	Inherent, Residual, Target and annual/as-needed assessment	three score types and assessment history
pp. 11-12	control documentation and control types	structured manual Control fields
p. 12	action plans and action owner	Remediation Action fields/status/deadline/owner
p. 12	risk acceptance reasons, considerations, six-month validity	Acceptance form and six-month business rule
p. 13	monitoring, annual review, emerging risk, top risk reporting	review date, trend/outlook, Dashboard/Reports
p. 14	Risk Owner, Control Owner, Action Owner duties	record-level role behavior and owner-specific edit scope

Not fully implemented from Framework

- Risk Appetite Statements/KRIs and RAG thresholds are not a dedicated module;
- formal committee approval/escalation workflow is not implemented;
- annual review reminders are not automated;
- Global Risk Sponsor/Risk Leader roles are not separate system roles;
- evidence attachments are not stored.

3. Risk Rating Matrix 2026

Source location	Rule	AIZEN implementation
p. 1	Risk Rating = Impact x Likelihood	common risk engine
pp. 2-5	Financial, Reputational, Operational, Regulatory/Legal Impact; highest applicable impact	Impact guidance shown/documented; user selects final 1-5
p. 5	Minor/Moderate/Major/Severe/Critical = 1-5	assessment labels
p. 6	next 12 months likelihood; Remote to Almost Certain with probability bands	dropdown labels/guidance
p. 7	exact 5x5 colored matrix	25 configurable cells and defaults
p. 7	2x3 = 6 Medium; 5x4 = 20 Significant examples	calculation tests/documentation
p. 8	Low/Medium/High/Significant response guidance	rating labels and dashboard/filter semantics

As-built nuance

Risk stores only Impact/Likelihood. Score/rating are recalculated from the current configured matrix. Historical matrix version pinning is a Phase 2 requirement.

4. LEG-POL-001 - Georgian Risk Management Policy

Source location	Rule/terminology	AIZEN use
pp. 4-5	continuous identification; clear, short risk narrative	identification workflow
p. 5	Cause, Event, Consequence structure	three mandatory description fields
pp. 5-6	risk categories	bilingual category labels/background context
pp. 7-9	Inherent/Current/Target assessment and likelihood x impact	Georgian UI terminology; calculation superseded by 2026 horizon
pp. 9-10	Avoid, Reduce/Mitigate, Transfer, Accept	Risk Response options
pp. 10-11	acceptance rationale and approval/escalation	Acceptance form and Phase 2 workflow requirement
pp. 11-12	monitoring, reporting, escalation, Three Lines	audit/reporting/role context
pp. 13-14	register field descriptions, risk ID, trend and outlook	Risk Register columns, ID format, Trend/Outlook separation
p. 17	business/function code + 001 sequence	Business Unit prefix-based risk reference generation

Translation note

The product PRD treats LEG-POL-001 primarily as Georgian terminology/label source. Company-specific references and old thresholds are not silently converted into current AIZEN business rules.

5. Request for the change.pdf

Page	Requested change	Implemented logic
1	remove Saved Filter; save columns and Compact/Detailed; set default	Saved View full lifecycle, user-specific default
2	equal-sized Inherent/Residual/Target chips	common chip dimensions and centered text
3	AD/OU-style Business Unit tree linked to permissions	parent/child hierarchy, descendant inheritance, no sibling/upward access

6. Request for change 2.pdf

Page	Requested change	Implemented logic
1	larger readable rating chips in Detailed view	full label/score/Impact x Likelihood presentation
2	Overview must show controls and action description/deliverable/status/deadline	Overview control summary and action table
3	register header font +25%	detailed register header sizing in current UI
4	Dashboard creation from Risk Register data; color/save	Dashboard Builder with filters, widgets, colors, persistence
5	Report templates with dashboards, open text and selectable compact register columns	Report Template Builder and three section types

7. Presentation2.pdf

Presentation2 mainly governs the public AIZEN website rather than the Risk/Administration modules. Relevant cross-product requirements are:

Page	Requirement	Product behavior
1	smooth 3-second Dashboard/Register/Reporting carousel with anonymized data	public-site Hero Carousel
2	solution descriptions	public-site solution cards
3-4	separate About Us + Team Members; image crop/focus	Website Administration (Super Admin only)
5	smooth page transitions	shared route transition/reduced-motion behavior

Website Administration is deliberately separate from Risk Administration.

8. Risk Category Library 2026.xlsx

The category library seeds a two-level editable category structure. In the referenced build it is represented as five Level 1 groups and 38 Level 2 categories. Administration can add/edit/deactivate categories; existing records retain references to inactive categories.

Note: the full category row-by-row content is master data, not reproduced in this functional-logic manual. It should remain available as seed/import data and receive separate data-quality review.

9. As-built AIZEN 4.1.0 Code

Area	As-built behavior source
Local persistence	LocalStorageRepository, storage key erm-risk-management-v3-state, schema 8
Future API	ApiRepository scaffold / Repository Pattern
risk visibility	named role checks + Business Unit effective scope fallback
permission aggregation	highest None/Read/Edit per module across roles
score/rating	common exact-cell risk engine
trend	last two residual assessment snapshots
actions overdue	date passed and not Completed
saved views	filters + columns + mode + default + user ID
dashboards	seven widget types and seven filters
reports	Dashboard/Open Text/Compact Register sections
admin	ten sections with Categories/BUs/Attributes/Users/Roles/Matrix/Branding/SSO/Data
audit	mutable summary event model in local state

10. Requirement Status Legend

Status	Meaning
Implemented - Phase 1	current offline build behavior
Business Rule	must remain true independent of frontend/backend
Phase 2 Requirement	must be server-side/enforced in production
Known Limitation	current intentional/technical gap
Source-derived	explicitly supported by uploaded source
As-built detail	observed in current implementation, not necessarily prescribed by policy

11. Open Product Decisions

The uploaded sources do not fully define the following. They require Product/Risk approval before backend implementation:

1. exact status transition matrix for Risk, Assessment and Acceptance;
2. approval authorities by rating/Business Unit;
3. whether a Completed action must always have progress = 100;
4. whether Residual may be lower than Target or Inherent in all cases;
5. whether Risk Owner may create controls/actions or only update them;
6. multiple-role record visibility union rules;
7. category/attribute deletion and remapping policy;
8. risk register uniqueness rule for titles;
9. evidence attachment taxonomy and retention;
10. KRI/Risk Appetite module scope;
11. reporting period freeze/snapshot rules;
12. notification/reminder frequency.

Until approved, the manual documents current Phase 1 behavior and labels recommended Phase 2 controls separately.

